AbandonedShipCard

+ AbandonedShipCard(Game, Level, int, int, int, int):

- accepted: boolean

+ initialize(): void
+ getReward(): void

accepted: boolean
nextState: AdventureState

AbandonedShipCard

+ AbandonedShipCard(Level, int, int, int, int):

- flightDaysLoss: int
- requiredCrew: int
- creditPrize: int

creditPrize: int
flightDaysLoss: int
requiredCrew: int

AbandonedStationCard

+ AbandonedStationCard(Game, Level, Map<GoodsType, Integer>, in

+ initialize(): void
# getReward(): void

nextState: AdventureState

AbandonedStationCard

+ AbandonedStationCard(Level, Map<GoodsType, Integer>, int, int, in

- requiredCrew: int
- flightDaysLoss: int
- goodsPrize: Map<GoodsType, Integer>

flightDaysLoss: int
requiredCrew: int
goodsPrize: Map<GoodsType, Integer>

AcquireForecast

+ AcquireForecast(int):

+ execute(VirtualServer): void

<<interface>>
Activatable

active: boolean

<<interface>>
Activatable

+ activate(): void
+ deactivate(): void

active: boolean

<<record>>
ActivatablePayload

+ ActivatablePayload(boolean):

+ active(): boolean

ActivateComponent

+ ActivateComponent(Point):

+ execute(VirtualServer): void

<<record>>
ActivateComponentEvent

+ ActivateComponentEvent(String, Point, boolean):

+ active(): boolean
+ playerName(): String
+ i t() P i t

ActivateState

# ActivateState(ShipBoard, Set<Point>):

# shipBoard: ShipBoard
# availablePositions: Set<Point>

+ skip(ShipBoard): void
+ spendBatteries(ShipBoard, Point): void
# endStateAction(): void
+ goNext(ShipBoard): void
+ activateComponent(ShipBoard, Point): void

availablePositions: Set<Point>
shipBoard: ShipBoard

ActivateState

~ ActivateState(ShipBoard, ShipBoard, Set<Point>):

# availablePositions: Set<Point>

+ notifyUseBattery(ShipBoard, Point): void
+ notifyActivateComponent(ShipBoard, Point): void

availableActions: List<StateActions>
availablePositions: Set<Point>

<<record>>
ActiveLobbyDTO

+ ActiveLobbyDTO(UUID, Level, int, List<String>, String):

+ level(): Level
+ numPlayers(): int
+ id(): UUID
+ players(): List<String>
+ host(): String

<<record>>
AddActiveLobbyEvent

+ AddActiveLobbyEvent(ActiveLobbyDTO):

+ newLobby(): ActiveLobbyDTO

<<record>>
AddGoodsDTO

+ AddGoodsDTO(String, Map<GoodsType, Integer>):

+ playerName(): String
+ goodsBuffer(): Map<GoodsType, Integer>

AddGoodsState

+ AddGoodsState(ShipBoard, Map<GoodsType, Integer>, ShipBoard):

- goodsBuffer: GoodsBuffer

+ notifyRemoveGoods(ShipBoard, Point, GoodsType): void
+ notifyPlaceGoods(ShipBoard, Point, GoodsType): void

availableActions: List<StateActions>
goodsBuffer: GoodsBuffer

AddGoodsState

+ AddGoodsState(Map<GoodsType, Integer>, ShipBoard):

~ goodsBuffer: Map<GoodsType, Integer>
~ shipBoard: ShipBoard

+ skip(ShipBoard): void
+ addGood(ShipBoard, Point, GoodsType): void
+ removeGood(ShipBoard, Point, GoodsType): void
+ goNext(ShipBoard): void

goodsBuffer: Map<GoodsType, Integer>
shipBoard: ShipBoard

AdventureCard

# AdventureCard(Game, Level, int):

- cardLevel: Level
- id: int

+ initialize(): void

nextState: AdventureState
cardLevel: Level
id: int

AdventureCard

# AdventureCard(Level, int):

- id: int
- cardLevel: Level

cardLevel: Level
id: int

<<interface>>
AdventureCardInterface

AdventureCardRegistry

# AdventureCardRegistry():

- instance: AdventureCardRegistry

# parseAdventureCard(JsonNode): AdventureCard
# loadRelevantCards(): void
# parsePlanets(JsonNode): List<Map<GoodsType, Integer>>
# parseChecks(JsonNode): List<CombatZoneCheck>
+ getCard(int): AdventureCard
# parseGoods(JsonNode): Map<GoodsType, Integer>
# parseProjectiles(JsonNode): List<Projectile>
# parsePenalties(JsonNode, JsonNode): List<Penalty>

instance: AdventureCardRegistry
size: int
idToImagePath: Map<Integer, Path>

AdventureState

+ AdventureState():

# imOut: boolean

+ notifyLoseCrew(ShipBoard, Point): void

availableActions: List<StateActions>
game: Game
imOut: boolean
shipBoard: ShipBoard
currentCard: AdventureCard

AdventureState

+ AdventureState():

# getNextState(): void
# checkIfExpired(): void
+ giveUp(ShipBoard): void

<<interface>>
AdventureStateInterface

Battery

+ Battery(Map<Direction, Connector>, int, int):

~ numBatteries: int

+ removeBattery(): void

numBatteries: int

Battery

+ Battery(Map<Direction, Connector>, int, int):
+ Battery(Map<Direction, Connector>, int):
+ Battery(int):

~ numBatteries: int

+ useBatteries(): void

numBatteries: int

<<record>>
BatteryPayload

+ BatteryPayload(int):

+ numBatteries(): int

BigFire

+ BigFire(IntSupplier, Direction):
+ BigFire(Direction):

# getComponentPositionToRemove(ShipBoard): Optional<Point>
+ getActivatablePoints(ShipBoard): Set<Point>

projectileType: ProjectileType

BigMeteor

+ BigMeteor(IntSupplier, Direction):
+ BigMeteor(Direction):

+ getActivatablePoints(ShipBoard): Set<Point>
# getComponentPositionToRemove(ShipBoard): Optional<Point>
# cannonPositionIsEffective(Entry<Point, Cannon>): boolean

projectileType: ProjectileType

<<record>>
BuildingDataDTO

+ BuildingDataDTO(int, List<Integer>, Set<String>):

+ uncoveredIds(): List<Integer>
+ numCovered(): int
+ l t dN () S t<St i >

Cabin

+ Cabin(Map<Direction, Connector>, int):

- crewType: CrewType
- numResidents: int

+ initialize(CrewType): void
+ loseCrew(): void

numResidents: int
crewType: CrewType

Cabin

+ Cabin(Map<Direction, Connector>, int):
+ Cabin():
+ Cabin(Map<Direction, Connector>):

- crewType: CrewType
- numResidents: int

+ loseResidents(): void
+ initialize(CrewType): void

numResidents: int
crewType: CrewType

<<record>>
CabinPayload

+ CabinPayload(CrewType, int):

+ crewType(): CrewType
+ numResidents(): int

Cannon

+ Cannon(Map<Direction, Connector>, int):

firePower: int

Cannon

+ Cannon(Map<Direction, Connector>):
+ Cannon(Map<Direction, Connector>, int):
+ Cannon():

firePower: int

CargoHold

+ CargoHold(Map<Direction, Connector>, int, boolean, int):
+ CargoHold(Map<Direction, Connector>, int, boolean):
+ CargoHold(int):

- goods: Map<GoodsType, Integer>

+ removeGoods(GoodsType, int): void
+ addGoods(GoodsType, int): void

isSpecial: boolean
goods: Map<GoodsType, Integer>

CargoHold

+ CargoHold(Map<Direction, Connector>, int, Boolean, int):

- size: int
- isSpecial: boolean
- goods: Map<GoodsType, Integer>

+ addGoods(GoodsType): void
+ removeGoods(GoodsType): void

size: int
isSpecial: boolean
goods: Map<GoodsType, Integer>

<<record>>
CargoPayload

+ CargoPayload(Map<GoodsType, Integer>):

+ goods(): Map<GoodsType, Integer>

CheatCodes

+ CheatCodes():

+ cheat(): String
+ cheatEmpty(): boolean
+ activateCheats(int): void

cheats: boolean
cheatOn: boolean

ChoosePlanet

+ ChoosePlanet(int):

+ execute(VirtualServer): void

<<record>>
ChoosePlanetDTO

+ ChoosePlanetDTO(String, int):

+ playerName(): String
+ numPlanets(): int

ChoosePlanetState

+ ChoosePlanetState(BiConsumer<ShipBoard, Integer>, int):

- numPlanets: int
- chosenPlanets: ShipBoard[]

+ skip(ShipBoard): void
+ goNext(ShipBoard): void
- nextShip(): ShipBoard?
+ choosePlanet(ShipBoard, int): void

currentShip: ShipBoard
game: Game
numPlanets: int
chosenPlanets: ShipBoard[]

ChoosePlanetState

+ ChoosePlanetState(ShipBoard, ShipBoard, int):

- options: ShipBoard[]
- isMyTurn: boolean

+ notifyCurrentPlayerUpdate(ShipBoard): void
+ notifyChoosePlanet(ShipBoard, int, ShipBoard): void

availableActions: List<StateActions>
isMyTurn: boolean
options: ShipBoard[]

ChooseShipPiece

+ ChooseShipPiece(int):

+ execute(VirtualServer): void

<<record>>
ChooseShipPieceDTO

+ ChooseShipPieceDTO(String, List<Set<Point>>):

+ shipPieces(): List<Set<Point>>
+ playerName(): String

ChooseShipPieceState

+ ChooseShipPieceState(ShipBoard, List<Set<Point>>, ShipBoard):

- shipPieces: List<Set<Point>>

+ notifyChooseShipPiece(ShipBoard, int): void

availableActions: List<StateActions>
shipPieces: List<Set<Point>>

ChooseShipPieceState

+ ChooseShipPieceState(List<Set<Point>>, ShipBoard):

- shipBoard: ShipBoard
- shipPieces: List<Set<Point>>

+ chooseShipPiece(ShipBoard, int): void
+ skip(ShipBoard): void

shipPieces: List<Set<Point>>
shipBoard: ShipBoard

CircularToggleButton

+ CircularToggleButton(Color):

- active: BooleanProperty

- toHex(Color): String
+ isActiveProperty(): BooleanProperty
- updateStyle(boolean): void

active: boolean
color: Color

CliActivationScreen

+ CliActivationScreen(ClientModel, ClientControllerInterface, Activ

+ parseAndInvoke(String): void
+ notifyUseBattery(ShipBoard, Point): void
+ notifyActivateComponent(ShipBoard, Point): void

CliAdventureCard

+ CliAdventureCard(AdventureCard):

- describeGoods(Map<GoodsType, Integer>): List<String>
- describePlanets(List<Map<GoodsType, Integer>>): List<String>
- describeProjectiles(List<Projectile>): List<String>

newDescription: List<String>

CliAdventureScreen

+ CliAdventureScreen(ClientModel, ClientControllerInterface, Adven

# printCurrentCard(): List<String>

CliAllShips

+ CliAllShips(List<CliShipBoard>):

newDescription: List<String>

CliBattery

+ CliBattery(Battery):

newDescription: List<String>

CliCabin

+ CliCabin(Cabin):

newDescription: List<String>

CliCannon

+ CliCannon(Cannon):

newDescription: List<String>

CliCargoHold

+ CliCargoHold(CargoHold):

newDescription: List<String>

CliComponent

# CliComponent(Component):

+ getConnector(Direction): String
+ highlight(CliHighlights): void
+ of(Component): CliComponent
# addBorders(String): List<String>

newDescription: List<String>
id: int

CliComponentBank

+ CliComponentBank(ComponentBank):

- uncoveredComponents: List<CliComponent>
- coveredComponentsN: int

+ addUncovered(Component): void
+ removeUncovered(Component): void
+ removeCovered(): void

coveredComponentsN: int
newDescription: List<String>
uncoveredComponents: List<CliComponent>

CliComponentLegend

+ CliComponentLegend():

- formatComponentName(String): String

newDescription: List<String>

CliCrewInitializationScreen

+ CliCrewInitializationScreen(ClientModel, ClientControllerInterfa

+ render(): void
+ parseAndInvoke(String): void
+ notifyInitializeCabin(ShipBoard, Point, CrewType, int): void

CliDeclareEnginePowerScreen

+ CliDeclareEnginePowerScreen(ClientModel, ClientControllerInterfa

+ render(): void

CliDeclareFirePowerScreen

+ CliDeclareFirePowerScreen(ClientModel, ClientControllerInterface

+ render(): void

CliDoubleCannon

+ CliDoubleCannon(DoubleCannon):

newDescription: List<String>

CliDoubleEngine

# CliDoubleEngine(DoubleEngine):

newDescription: List<String>

CliElement

+ CliElement():

+ setDirty(): void

newDescription: List<String>
description: List<String>

CliEndGameScreen

+ CliEndGameScreen(ClientModel, ClientControllerInterface):

+ isInputLegal(String): boolean
+ parseAndInvoke(String): void
+ render(): void

CliEngine

+ CliEngine(Engine):

newDescription: List<String>

CliFlightBoard

+ CliFlightBoard(FlightBoard):

+ setPosition(ShipBoard, int): void

newDescription: List<String>

CliForecast

+ CliForecast(ShipBoard[]):

+ removeBlockedForecast(int): void
+ setBlockedForecasts(int, GameColor): void

newDescription: List<String>

CliForecastCards

+ CliForecastCards():

newDescription: List<String>
cards: List<AdventureCard>

CliGameCreationScreen

+ CliGameCreationScreen(ClientModel, ClientControllerInterface):

+ render(): void
+ parseAndInvoke(String): void
+ isInputLegal(String): boolean

CliGoodsScreen

+ CliGoodsScreen(ClientModel, ClientControllerInterface, AddGoodsS

- countCargoHoldsWithGoods(): int
+ parseAndInvoke(String): void
+ render(): void

CliHand

+ CliHand():
+ CliHand(Component):

+ clearHand(): void

newDescription: List<String>
hand: Component

<<enumeration>>
CliHighlights

- CliHighlights(String):

+ valueOf(String): CliHighlights
+ getSomeColors(int): List<CliHighlights>
+ values(): CliHighlights[]

CliJoinOrCreateScreen

+ CliJoinOrCreateScreen(ClientModel, ClientControllerInterface):

+ notifyRemoveLobby(UUID): void
+ isInputLegal(String): boolean
+ render(): void
+ parseAndInvoke(String): void
+ notifyNewLobby(Lobby): void

CliLifeSupport

+ CliLifeSupport(LifeSupport):

newDescription: List<String>

CliLobbyScreen

+ CliLobbyScreen(ClientModel, ClientControllerInterface):

+ render(): void
+ parseAndInvoke(String): void
+ isInputLegal(String): boolean

CliLoseGoodsScreen

+ CliLoseGoodsScreen(ClientModel, ClientControllerInterface, Remov

+ render(): void
+ parseAndInvoke(String): void

CliNewCardScreen

+ CliNewCardScreen(ClientModel, ClientControllerInterface, DrawCar

+ render(): void
+ parseAndInvoke(String): void
+ notifyDrawCard(AdventureCard): void

CliNicknameChoiceScreen

+ CliNicknameChoiceScreen(ClientModel, ClientControllerInterface):

+ render(): void
+ isInputLegal(String): boolean
+ parseAndInvoke(String): void

CliPlanetScreen

+ CliPlanetScreen(ClientModel, ClientControllerInterface, ChooseP

+ render(): void
+ notifyPlaceGoods(ShipBoard, Point, GoodsType): void
+ parseAndInvoke(String): void
+ notifyCurrentPlayerUpdate(ShipBoard): void
+ notifyChoosePlanet(ShipBoard, int, ShipBoard): void

CliProjectilesScreen

+ CliProjectilesScreen(ClientModel, ClientControllerInterface, Ha

+ render(): void
+ notifyRemoveComponent(ShipBoard, Point): void

CliRemoveCrewScreen

+ CliRemoveCrewScreen(ClientModel, ClientControllerInterface, Remo

+ render(): void
+ parseAndInvoke(String): void

CliRewardScreen

+ CliRewardScreen(ClientModel, ClientControllerInterface, GrabRewa

+ render(): void
+ parseAndInvoke(String): void

CliScreen

+ CliScreen(ClientModel, ClientControllerInterface):
+ CliScreen(ClientModel, ClientControllerInterface, GameState):

# state: GameState

+ render(): void
# getPoint(String): Point
+ notifyGiveUp(Player): void
+ notifyComponentChange(ShipBoard, Point): void
# printShipFlightStats(): List<String>
+ parseAndInvoke(String): void
# isFormatLegal(String): boolean
# printActions(): void
+ isInputLegal(String): boolean

state: GameState

CliScreenFactory

+ CliScreenFactory():

+ createScreen(GameState, ClientModel, ClientControllerInterface): CliScreen
+ createScreen(MetaState, ClientModel, ClientControllerInterface): CliScreen

CliSecondShipBuildingScreen

+ CliSecondShipBuildingScreen(ClientModel, ClientControllerInterface,

+ notifyRequestComponent(ShipBoard, Component): void
+ notifyPlaceComponent(ShipBoard, Point, Direction, Point): void
+ notifyRejectComponent(ShipBoard, Component, Point): void
+ notifyPlaceComponent(ShipBoard, Point, Direction): void
+ notifyReleaseForecast(ShipBoard, int): void
+ notifyStashComponent(ShipBoard, Component): void
+ notifyGrabPlacedComponent(ShipBoard, Point): void
+ render(): void
+ notifyFlightBoardPosition(ShipBoard, int): void
+ notifyRequestRandComponent(ShipBoard, Component): void
+ notifyRejectComponent(ShipBoard, Component): void
+ parseAndInvoke(String): void
+ notifyFlipHourglass(ShipBoard): void
+ notifyGrabStashedComponent(ShipBoard, int, Component): void
- componentInHand(): boolean
+ notifyStashComponent(ShipBoard, Component, Point): void
+ notifyPeekForecast(ShipBoard, int): void

forecastDeck: List<AdventureCard>

CliShield

+ CliShield(Shield):

newDescription: List<String>

CliShipAndHand

+ CliShipAndHand(ShipBoard, String):

+ clearHand(): void

newDescription: List<String>
hand: Component

CliShipBoard

+ CliShipBoard(ShipBoard, String):

- nickname: String

+ highlightPoints(Set<Point>, CliHighlights): void
+ getCliComponent(Point): CliComponent
+ onPutComponent(Point, Component): void
+ onRemoveComponent(Point): void

newDescription: List<String>
nickname: String

CliShipHandAndStash

+ CliShipHandAndStash(ShipBoard, String):

+ clearHand(): void
+ onGrabStashed(int): void
+ onStash(Component): void

newDescription: List<String>
hand: Component
nickname: String

CliShipPieceChoiceScreen

+ CliShipPieceChoiceScreen(ClientModel, ClientControllerInterface,

- shipNotConnected(ShipBoard, List<Set<Point>>): void
+ parseAndInvoke(String): void
+ render(): void
+ notifyChooseShipPiece(ShipBoard, int, List<Point>): void

CliStash

+ CliStash(List<Component>):

+ onStash(Component): void
+ onGrab(int): void

newDescription: List<String>

CliTestShipBuildingScreen

+ CliTestShipBuildingScreen(ClientModel, ClientControllerInterface, Te

+ notifyFlightBoardPosition(ShipBoard, int): void
+ parseAndInvoke(String): void
+ notifyRejectComponent(ShipBoard, Component, Point): void
- componentInHand(): boolean
+ notifyGrabPlacedComponent(ShipBoard, Point): void
+ notifyPlaceComponent(ShipBoard, Point, Direction): void
+ notifyPlaceComponent(ShipBoard, Point, Direction, Point): void
+ notifyRequestComponent(ShipBoard, Component): void
+ render(): void
+ notifyRejectComponent(ShipBoard, Component): void
+ notifyRequestRandComponent(ShipBoard, Component): void

CliValidationScreen

+ CliValidationScreen(ClientModel, ClientControllerInterface, ShipCo

+ notifyShipValidated(ShipBoard): void
+ notifyChooseShipPiece(ShipBoard, int, List<Point>): void
+ notifyShipNotConnected(ShipBoard, List<Set<Point>>): void
+ parseAndInvoke(String): void
+ render(): void
+ notifyRemoveComponent(ShipBoard, Point): void
- shipNotConnected(ShipBoard, List<Set<Point>>): void

CliView

+ CliView(ClientController, ClientModel):

+ reportError(String): void
+ notifyLoseCrew(ShipBoard, Point): void
+ notifyRejectComponent(ShipBoard, Component, Point): void
+ notifyPlayerJoin(Player): void
+ notifyStashComponent(ShipBoard, Component, Point): void
+ notifyRemoveLobby(UUID): void
+ notifyInitializeCabin(ShipBoard, Point, CrewType, int): void
+ notifyRequestComponent(ShipBoard, Component): void
+ notifyReleaseForecast(ShipBoard, int): void
+ notifyNewLobby(Lobby): void
+ notifyGrabReward(ShipBoard, boolean): void
+ notifyFlipHourglass(ShipBoard): void
+ notifyDrawCard(AdventureCard): void
+ notifyRequestRandComponent(ShipBoard, Component): void
+ notifyGrabPlacedComponent(ShipBoard, Point): void
+ notifyPlaceComponent(ShipBoard, Point, Direction): void
+ notifyShipValidated(ShipBoard): void
+ notifyChoosePlanet(ShipBoard, int, ShipBoard): void
+ notifyFlightBoardPosition(ShipBoard, int): void
+ notifyCurrentState(GameState): void
+ notifyUseBattery(ShipBoard, Point): void
+ notifyCurrentPlayerUpdate(ShipBoard): void
- startInputLoop(): void
+ notifyHourglassEnd(): void
+ notifyShipNotConnected(ShipBoard, List<Set<Point>>): void
+ notifyActivateComponent(ShipBoard, Point): void
+ notifyPlaceComponent(ShipBoard, Point, Direction, Point): void
+ notifyStashComponent(ShipBoard, Component): void
+ notifyRemoveGoods(ShipBoard, Point, GoodsType): void
+ notifyPlaceGoods(ShipBoard, Point, GoodsType): void
+ notifyPeekForecast(ShipBoard, int): void
+ notifyChooseShipPiece(ShipBoard, int, List<Point>): void
+ notifyGiveUp(Player): void
+ notifyGrabStashedComponent(ShipBoard, int, Component): void
+ notifyRemoveComponent(ShipBoard, Point): void
+ notifyMetaState(MetaState): void
+ notifyRejectComponent(ShipBoard, Component): void

forecastDeck: List<AdventureCard>
finalScores: Map<Player, Integer>

Client

+ Client():

+ launchUI(): void
+ connect(String, String, int, int): void
+ main(String[]): void

ClientController

+ ClientController():

- server: ServerHandler
- view: ErrorReporter
- model: ClientModel

+ signalDisconnection(): void
+ placeGoods(Point, GoodsType): void
+ placeShipOnFlightboard(int): void
+ requestRandComponent(): void
+ requestNewGame(Level, int): void
+ clearNickname(): void
+ loseCrew(Point): void
+ grabStashedComponent(int): void
+ placeComponent(Point, Direction): void
+ choosePlanet(int): void
+ registerNickname(String): void
+ initializeCabin(Point, CrewType): void
+ initEventHandler(): void
+ showGameCreation(): void
- reconnect(): void
+ reportError(String): void
+ drawCard(): void
+ activateComponent(Point): void
+ removeComponent(Point): void
+ acquireForecast(int): void
+ dropConnection(): void
+ joinLobby(UUID): void
+ grabPlacedComponent(): void
+ flipHourglass(): void
+ requestComponent(int): void
+ grabReward(): void
+ rejectComponent(): void
+ stashComponent(): void
+ releaseForecast(): void
+ removeGoods(Point, GoodsType): void
+ clearModel(): void
+ giveUp(): void
+ chooseShipPiece(int): void
+ quit(): void
+ goNext(): void
+ loseGoods(Point): void
+ useBattery(Point): void
+ placeShipOnFlightBoard(): void
+ notifyEvent(Event): void

server: ServerHandler
myNickname: String
view: ErrorReporter
model: ClientModel

<<interface>>
ClientControllerInterface

+ initializeCabin(Point, CrewType): void
+ requestRandComponent(): void
+ choosePlanet(int): void
+ drawCard(): void
+ joinLobby(UUID): void
+ chooseShipPiece(int): void
+ rejectComponent(): void
+ grabPlacedComponent(): void
+ removeGoods(Point, GoodsType): void
+ quit(): void
+ flipHourglass(): void
+ placeGoods(Point, GoodsType): void
+ grabReward(): void
+ activateComponent(Point): void
+ giveUp(): void
+ removeComponent(Point): void
+ requestNewGame(Level, int): void
+ stashComponent(): void
+ acquireForecast(int): void
+ placeComponent(Point, Direction): void
+ registerNickname(String): void
+ requestComponent(int): void
+ useBattery(Point): void
+ grabStashedComponent(int): void
+ goNext(): void
+ loseGoods(Point): void
+ placeShipOnFlightboard(int): void
+ loseCrew(Point): void
+ showGameCreation(): void
+ placeShipOnFlightBoard(): void
+ clearModel(): void
+ releaseForecast(): void

ClientEventHandler

+ ClientEventHandler(ClientController, ClientModel, PlayerRegistry

- updateGameState(StateDTO): void
+ handleEvent(Event): void
- updateGameState(ComplexStateDTO): void
- setupLobby(LobbyDetailsDTO, boolean): void

ClientEventQueue

+ ClientEventQueue():

+ notifyEvent(Event): void
+ pause(): void

<<interface>>
ClientHandler

+ pauseEvents(): void
+ stop(): void

player: Player

ClientModel

+ ClientModel():

- clientPlayer: Player
- game: Game
- activeLobbies: Map<UUID, Lobby>
- finalScores: Map<Player, Integer>
- shipToPlayer: Map<ShipBoard, Player>
- players: Set<Player>
- metaState: MetaState

+ notifyGrabCredits(ShipBoard, int): void
+ addPlayer(Player, GameColor): void
+ notifyHourglassEnd(): void
+ notifyReleaseForecast(ShipBoard): void
+ notifyShipValidated(ShipBoard): void
+ activateCheats(int): void
+ notifyStashComponent(ShipBoard): void
+ notifyChooseShipPiece(ShipBoard, int): void
+ notifyPlaceComponent(ShipBoard, Point, Direction): void
+ notifyDrawCard(AdventureCard): void
+ getPlayerByShip(ShipBoard): Player
+ notifyFlightBoardPosition(ShipBoard, int): void
+ notifyPeekForecast(ShipBoard, int): void
+ notifyRequestComponent(ShipBoard, Component): void
+ notifyRemoveComponent(ShipBoard, Point): void
+ notifyRejectComponent(ShipBoard): void
+ notifyRemoveGoods(ShipBoard, Point, GoodsType): void
+ notifyNewLobby(Lobby): void
+ notifyRemoveLobby(UUID): void
+ notifyCurrentPlayerUpdate(ShipBoard): void
+ addObserver(ModelObserver): void
+ notifyGrabStashedComponent(ShipBoard, int): void
+ notifyGrabPlacedComponent(ShipBoard): void
+ notifyGiveUpMessage(ShipBoard): void
+ notifyLoseCrew(ShipBoard, Point): void
+ notifySurrenderShip(Set<ShipBoard>): void
+ notifyRequestRandComponent(ShipBoard, Component): void
+ clearGame(): void
+ createGame(Level): void
+ notifyCurrentState(GameState): void
+ notifyPlaceGoods(ShipBoard, Point, GoodsType): void
+ notifyInitializeCabin(ShipBoard, Point, CrewType): void
+ notifyChoosePlanet(ShipBoard, int, ShipBoard): void
+ notifyActivateComponent(ShipBoard, Point): void
+ notifyShipNotConnected(ShipBoard, List<Set<Point>>): void
+ notifyUseBattery(ShipBoard, Point): void
+ notifyFlipHourglass(ShipBoard): void
+ removeObserver(ModelObserver): void
+ notifySurrenderRequest(Player): void
- safeGetCurrentState(): GameState

clientPlayer: Player
myShip: ShipBoard
players: Set<Player>
activeLobbies: Map<UUID, Lobby>
player: Player
metaState: MetaState
game: Game
forecastDeck: List<AdventureCard>
finalScores: Map<Player, Integer>
shipToPlayer: Map<ShipBoard, Player>

CombatZoneCard

+ CombatZoneCard(Game, Level, List<CombatZoneCheck>, List<Penalt

- checks: List<CombatZoneCheck>
- penalties: List<Penalty>

+ initialize(): void

checks: List<CombatZoneCheck>
penalties: List<Penalty>
nextState: AdventureState

CombatZoneCard

+ CombatZoneCard(Level, List<CombatZoneCheck>, List<Penalty>, int):

- penalties: List<Penalty>
- checks: List<CombatZoneCheck>

checks: List<CombatZoneCheck>
penalties: List<Penalty>

<<interface>>
CombatZoneCheck

+ getWeakestPlayer(FlightBoard): ShipBoard
+ getAvailableAction(ShipBoard): Optional<AdventureState>

<<enumeration>>
CombatZoneCheck

+ CombatZoneCheck():

+ values(): CombatZoneCheck[]
+ valueOf(String): CombatZoneCheck

<<record>>
ComplexChoosePlanetDTO

+ ComplexChoosePlanetDTO(ChoosePlanetDTO, String[]):

+ baseData(): ChoosePlanetDTO
+ choices(): String[]

<<record>>
ComplexDrawCardDTO

+ ComplexDrawCardDTO(String, boolean):

+ playerName(): String
+ hasDrawn(): boolean

<<interface>>
ComplexStateDTO

Component

+ Component(Map<Direction, Connector>, int):

- orientation: Direction
- connectors: Map<Direction, Connector>
- id: int

+ getRotatedConnector(Direction): Connector

connectors: Map<Direction, Connector>
orientation: Direction
id: int

Component

+ Component(Map<Direction, Connector>, int):
+ Component():
+ Component(Map<Direction, Connector>):

- orientation: Direction
- connectors: Map<Direction, Connector>
- id: int

connectors: Map<Direction, Connector>
orientation: Direction
id: int

ComponentBank

+ ComponentBank():

- coveredComponentsN: int
- uncoveredComponents: List<Component>

+ addUncoveredComponent(Component): void
+ removeCoveredComponent(): void
+ removeUncoveredComponent(Component): void

coveredComponentsN: int
uncoveredComponents: List<Component>

ComponentBank

+ ComponentBank():

- uncoveredComponents: Map<Integer, Component>
- coveredComponents: List<Component>

+ removeUncoveredComponent(int): Component
+ returnCoveredComponent(Component): void
+ drawRandComponent(): Component
+ initialize(): void
+ addToUncoveredComponents(Component): void

coveredComponents: List<Component>
numCovered: int
uncoveredIds: List<Integer>
uncoveredComponents: Map<Integer, Component>

<<record>>
ComponentDTO

+ ComponentDTO(int, Direction, ComponentPayload):

+ id(): int
+ orientation(): Direction
+ l d() C tP l d

<<interface>>
ComponentInterface

orientation: Direction
id: int

<<interface>>
ComponentPayload

+ numBatteries(): int
+ crewType(): CrewType
+ numResidents(): int
+ active(): boolean
+ goods(): Map<GoodsType, Integer>

ComponentRegistry

# ComponentRegistry():

- instance: ComponentRegistry

- parseComponent(JsonNode): Component
+ getStartingCabin(GameColor): Component
- parseCrewType(JsonNode): CrewType
- parseConnectors(JsonNode): Map<Direction, Connector>
# loadComponents(): void
+ getComponent(int): Component
- parseComponent(int, JsonNode): Component

size: int
idToImagePath: Map<Integer, Path>
instance: ComponentRegistry

ComponentRegistry

- ComponentRegistry():

- bankComponents: List<JsonNode>
- instance: ComponentRegistry

- parseConnectors(JsonNode): Map<Direction, Connector>
- loadFile(): void
+ getStartingCabin(GameColor): Component
+ getComponentById(int): Component
- parseCrewType(JsonNode): CrewType
- parseComponent(JsonNode): Component

instance: ComponentRegistry
bankComponents: List<Component>

<<enumeration>>
ComponentType

- ComponentType(List<String>):

+ getSymbol(int): String
+ valueOf(String): ComponentType
+ l () C tT []

<<enumeration>>
Connector

+ Connector():

+ values(): Connector[]
+ matches(Connector): boolean
+ l Of(St i) C t

<<interface>>
ControllerEvent

ControllerEventHandler

+ ControllerEventHandler(EventQueue<ControllerEvent>):

- sendEvent(ControllerEvent, String): void
- broadcastEvent(ControllerEvent): void
+ handleEvent(ControllerEvent): void

ConversionUtils

+ ConversionUtils():

+ convertArray(ShipBoard[]): String[]
+ convertCollection(T, Supplier<R>): R
+ convertMap(Map<ShipBoard, V>): Map<String, V>
+ convert(ShipBoard): String

ConversionUtils

+ ConversionUtils(PlayerRegistry):

# convertPlayerMap(Map<String, T>): Map<Player, T>
- getShipBuildingState(ClientModel): ShipBuildingState
- getHandleProjectileState(HandleProjectileDTO, ShipBoard): HandleProjectileState
- getChooseShipPieceState(ChooseShipPieceDTO, ShipBoard): ChooseShipPieceState
- getChoosePlanetState(ChoosePlanetDTO, ShipBoard): ChoosePlanetState
+ getGameState(ComplexStateDTO, ClientModel): GameState
- getRemoveCrewState(RemoveCrewDTO, ShipBoard): RemoveCrewState
- getSimpleState(SimpleStateDTO, ShipBoard): AdventureState?
# convertArray(String[]): ShipBoard[]
- getAddGoodsState(AddGoodsDTO, ShipBoard): AddGoodsState
# convertMap(Map<String, T>): Map<ShipBoard, T>
# convertCollection(T, Supplier<R>): R
- getShipInitializationState(ShipInitializationDTO, ShipBoard): ShipInitializationState
+ convertBuildingData(ShipBuildingState, BuildingDataDTO): void
+ getGameState(StateDTO, ClientModel): GameState
- getShipCorrectionState(ShipCorrectionDTO, ShipBoard): ShipCorrectionState
+ convertComponent(ComponentDTO): Component
# convertName(String): ShipBoard
- getRemoveGoodsState(RemoveGoodsDTO, ShipBoard): RemoveGoodsState

CrewLoss

+ CrewLoss(int):

~ crew: int

crew: int

CrewLoss

+ CrewLoss(int):

+ inflictPenalty(ShipBoard, FlightBoard): Optional<AdventureState>

CrewSizeCheck

- CrewSizeCheck():

- instance: CrewSizeCheck

+ getAvailableAction(ShipBoard): Optional<AdventureState>
+ getWeakestPlayer(FlightBoard): ShipBoard

instance: CrewSizeCheck

<<enumeration>>
CrewType

+ CrewType():

+ values(): CrewType[]
+ valueOf(String): CrewType

<<record>>
CurrentPlayerUpdateEvent

+ CurrentPlayerUpdateEvent(String):

+ playerName(): String

Deck

+ Deck(Game):

# masterDeck: List<AdventureCard>
- currentCard: AdventureCard

+ drawCard(): void
- parsePenalties(JsonNode, JsonNode): List<Penalty>
- parsePlanets(JsonNode): List<Map<GoodsType, Integer>>
- parseGoods(JsonNode): Map<GoodsType, Integer>
+ initMasterDeck(): void
# loadRelevantCards(Game): List<AdventureCard>
- parseChecks(JsonNode): List<CombatZoneCheck>
- parseProjectiles(JsonNode): List<Projectile>
+ getForecastDeck(int): List<AdventureCard>

empty: boolean
masterDeck: List<AdventureCard>
currentCard: AdventureCard

DeclareEnginePowerState

+ DeclareEnginePowerState(ShipBoard, ShipBoard):

DeclareEnginePowerState

+ DeclareEnginePowerState(ShipBoard):

DeclareFirePowerState

+ DeclareFirePowerState(ShipBoard, ShipBoard):

DeclareFirePowerState

+ DeclareFirePowerState(ShipBoard):

DescriptionUtils

+ DescriptionUtils():

+ sideBySide(List<String>, List<String>): List<String>
+ borderAndTitle(List<String>, String): List<String>
- getRealWidth(String): int
+ sideBySide(List<String>, List<String>, int): List<String>

Dice

+ Dice():

asInt: int

<<enumeration>>
Direction

+ Direction():

+ LEFT:
+ RIGHT:

+ rotateDirectionMap(Map<Direction, V>, Direction, Direction): Map<Direction, V>
+ getNeighbour(Point, Direction): Point
+ valueOf(String): Direction
+ values(): Direction[]
- computeSteps(Direction, Direction): int

LEFT: Direction
opposite: Direction
RIGHT: Direction
l i t

DoubleCannon

+ DoubleCannon(Map<Direction, Connector>):
+ DoubleCannon():
+ DoubleCannon(Map<Direction, Connector>, int):

- active: boolean

+ deactivate(): void
+ activate(): void

firePower: int
active: boolean

DoubleCannon

+ DoubleCannon(Map<Direction, Connector>, int):

- active: boolean

firePower: int
active: boolean

DoubleEngine

+ DoubleEngine(Map<Direction, Connector>):
+ DoubleEngine(Map<Direction, Connector>, int):
+ DoubleEngine():

- active: boolean

+ activate(): void
+ deactivate(): void

active: boolean
enginePower: int

DoubleEngine

+ DoubleEngine(Map<Direction, Connector>, int):

- active: boolean

active: boolean
enginePower: int

DrawCard

+ DrawCard():

+ execute(VirtualServer): void

DrawCardState

+ DrawCardState():

- shipBoard: ShipBoard

+ skip(ShipBoard): void
+ drawCard(ShipBoard): void
+ goNext(ShipBoard): void
+ hasDrawn(): boolean

game: Game
shipBoard: ShipBoard

DrawCardState

+ DrawCardState(ShipBoard, ShipBoard):

- hasDrawn: boolean

+ hasDrawn(): boolean
+ notifyDrawCard(AdventureCard): void

availableActions: List<StateActions>
hasDrawn: boolean

DtoConverter

+ DtoConverter():

- stateDtoOf(DeclareFirePowerState): SimpleStateDTO
- stateDtoOf(GrabRewardState): SimpleStateDTO
+ getActiveLobby(Lobby): ActiveLobbyDTO
- stateDtoOf(RemoveGoodsState): RemoveGoodsDTO
+ getHourglass(Hourglass): HourglassDTO
+ getShipBoard(ShipBoard): ShipBoardDTO
- stateDtoOf(RemoveCrewState): RemoveCrewDTO
- stateDtoOf(DeclareEnginePowerState): SimpleStateDTO
- stateDtoOf(ChoosePlanetState): ChoosePlanetDTO
+ getState(GameStateInterface): StateDTO
- stateDtoOf(ChooseShipPieceState): ChooseShipPieceDTO
- stateDtoOf(AddGoodsState): AddGoodsDTO
- stateDtoOf(HandleProjectileState): HandleProjectileDTO
+ getComplexState(GameStateInterface): ComplexStateDTO
- stateDtoOf(ShipCorrectionState): ShipCorrectionDTO
+ getFlightBoard(FlightBoard): FlightBoardDTO
- stateDtoOf(ShipInitializationState): ShipInitializationDTO
+ getLobbyDetails(Lobby): LobbyDetailsDTO
+ getBuildingData(ShipBuildingState): BuildingDataDTO
+ getComponent(ComponentInterface): ComponentDTO

EnabledSurrenderPolicy

+ EnabledSurrenderPolicy(GameEventListener):

- requests: Set<ShipBoard>
- surrenderedShips: Set<ShipBoard>
- listener: GameEventListener

+ confirmSurrender(FlightBoard): Set<ShipBoard>
+ requestSurrender(ShipBoard, SurrenderCause): boolean

requests: Set<ShipBoard>
surrenderedShips: Set<ShipBoard>
surrenderEnabled: boolean
listener: GameEventListener

<<record>>
EndByDisconnectionEvent

+ EndByDisconnectionEvent(String):

+ winnerName(): String

receiverName: Optional<String>

Engine

+ Engine(Map<Direction, Connector>):
+ Engine():
+ Engine(Map<Direction, Connector>, int):

enginePower: int
valid: boolean

Engine

+ Engine(Map<Direction, Connector>, int):

enginePower: int
valid: boolean

EnginePowerCheck

- EnginePowerCheck():

- instance: EnginePowerCheck

+ getAvailableAction(ShipBoard): Optional<AdventureState>
+ getWeakestPlayer(FlightBoard): ShipBoard

instance: EnginePowerCheck

EpidemicCard

+ EpidemicCard(Game, Level, int):

nextState: AdventureState

EpidemicCard

+ EpidemicCard(Level, int):

<<interface>>
ErrorReporter

+ reportError(String): void

<<interface>>
Event

+ shouldResume(): boolean

receiverName: Optional<String>

<<interface>>
EventHandler

+ handleEvent(T): void

<<interface>>
EventListener

+ notifyEvent(T): void

<<record>>
EventMessage

+ EventMessage(Event):

+ event(): Event

EventQueue

+ EventQueue():

+ notifyEvent(T): void
+ clear(): void
+ poll(): T

EventQueueHandler

+ EventQueueHandler(EventQueue<T>):

- afterEach: Runnable

- threadTask(): void
+ start(): void
+ stop(): void

afterEach: Runnable

FirePowerCheck

- FirePowerCheck():

- instance: FirePowerCheck

+ getAvailableAction(ShipBoard): Optional<AdventureState>
+ getWeakestPlayer(FlightBoard): ShipBoard

instance: FirePowerCheck

FlightBoard

+ FlightBoard(List<Integer>, int):

- loopLength: int
- startingPositions: List<Integer>
- shipToPlace: Map<ShipBoard, Integer>

+ removeShip(ShipBoard): void
+ setShipPosition(ShipBoard, int): void

startingPositions: List<Integer>
shipToPlace: Map<ShipBoard, Integer>
loopLength: int

FlightBoard

+ FlightBoard(GameEventListener):

# shipToPlace: Map<ShipBoard, Integer>
# startingPositionsLeft: List<Integer>

+ setup(Set<ShipBoard>): void
+ removeShips(Set<ShipBoard>): void
+ displaceShip(ShipBoard, int): void
+ placeShipOnFlightBoard(ShipBoard): void
+ placeShipOnFlightBoard(ShipBoard, int): void
- placeInternal(ShipBoard): int

lappedShips: Set<ShipBoard>
startingPositionsLeft: List<Integer>
shipToPlace: Map<ShipBoard, Integer>
loopLength: int
orderedShips: List<ShipBoard>

<<record>>
FlightBoardDTO

+ FlightBoardDTO(Map<String, Integer>):

+ playerToPlace(): Map<String, Integer>

<<record>>
FlightBoardUpdateEvent

+ FlightBoardUpdateEvent(String, int):

+ playerName(): String
+ position(): int

FlightDaysLoss

+ FlightDaysLoss(int):

FlightDaysLoss

+ FlightDaysLoss(int):

+ inflictPenalty(ShipBoard, FlightBoard): Optional<AdventureState>

FlipHourglass

+ FlipHourglass():

+ execute(VirtualServer): void

<<record>>
FlipHourglassEvent

+ FlipHourglassEvent(String):

+ playerName(): String

<<record>>
ForecastDetailsEvent

+ ForecastDetailsEvent(String, List<Integer>):

+ forecastDeckIds(): List<Integer>
+ playerName(): String

Game

+ Game(Level, int, GameEventListener):

- gameFactory: GameFactory
- currentState: GameState
- surrenderPolicy: SurrenderPolicy
- finalScores: Map<ShipBoard, Integer>
- flightBoard: FlightBoard
- level: Level
- eventListener: GameEventListener
- shipBoards: Set<ShipBoard>
- afterEach: Runnable
- scoresRegistry: ScoresRegistry
- gameOver: boolean
- deck: Deck

- withStateReadLock(Runnable): void
+ skipBuilding(): void
+ loseCrew(ShipBoard, Point): void
+ drawCard(ShipBoard): void
+ grabPlacedComponent(ShipBoard): void
+ tryEndGame(): boolean
+ submitStateTransition(Runnable): void
+ releaseForecast(ShipBoard): void
+ removeGoods(ShipBoard, Point, GoodsType): void
- assignShipRewards(): void
+ placeGoods(ShipBoard, Point, GoodsType): void
+ removeComponent(ShipBoard, Point): void
+ initializeCabin(ShipBoard, Point, CrewType): void
+ requestSnapshot(ShipBoard): void
+ loseGood(ShipBoard, Point): void
+ stashComponent(ShipBoard): void
+ addShipBoard(GameColor): ShipBoard
+ rejectComponent(ShipBoard): void
- withStateReadLock(Supplier<T>): T
+ placeShipOnFlightBoard(ShipBoard, int): void
- withStateWriteLock(Supplier<T>): T
+ grabReward(ShipBoard): void
+ acquireForecast(ShipBoard, int): void
+ goNext(ShipBoard): void
+ chooseShipPiece(ShipBoard, int): void
+ placeShipOnFlightBoard(ShipBoard): void
+ grabStashedComponent(ShipBoard, int): void
+ requestRandComponent(ShipBoard): void
+ giveUp(ShipBoard): void
+ skip(ShipBoard): void
- endGame(): void
+ requestComponent(ShipBoard, int): void
+ useBattery(ShipBoard, Point): void
+ start(): void
+ activateComponent(ShipBoard, Point): void
+ choosePlanet(ShipBoard, int): void
- withStateWriteLock(Runnable): void
+ placeComponent(ShipBoard, Point, Direction): void
- runRequest(Runnable): void
+ flipHourglass(ShipBoard): void

flightBoard: FlightBoard
afterEach: Runnable
shipBoards: Set<ShipBoard>
currentState: GameState
surrenderPolicy: SurrenderPolicy
gameOver: boolean
level: Level
deck: Deck
scoresRegistry: ScoresRegistry
startAdventureCallback: Runnable
eventListener: GameEventListener
gameFactory: GameFactory
finalScores: Map<ShipBoard, Integer>

Game

+ Game(Level):

- gameFactory: GameFactory
- flightBoard: FlightBoard
- observers: List<ModelObserver>
- currentCard: AdventureCard
- shipBoards: Set<ShipBoard>
- givenUpShips: Set<ShipBoard>
- myShip: ShipBoard
- level: Level
- currentState: GameState

+ addShipBoard(GameColor): ShipBoard

level: Level
flightBoard: FlightBoard
shipBoards: Set<ShipBoard>
myShip: ShipBoard
currentState: GameState
observers: List<ModelObserver>
gameFactory: GameFactory
shipColors: Set<GameColor>
currentCard: AdventureCard
givenUpShips: Set<ShipBoard>

<<enumeration>>
GameColor

+ GameColor():

+ values(): GameColor[]
+ valueOf(String): GameColor

GameColorCliMapper

+ GameColorCliMapper():

+ toAnsiBullet(GameColor): String

GameColorGuiMapper

+ GameColorGuiMapper():

+ toJfxColor(GameColor): Color

<<record>>
GameEndEvent

+ GameEndEvent(Map<String, Integer>):

+ playerToScore(): Map<String, Integer>
+ runLobbyAction(Lobby): void

GameEventListener

+ GameEventListener(EventListener<LobbyEvent>, Supplier<LobbyDetailsDTO

- startAdventureCallback: Runnable

+ notifyCurrentPlayerUpdateEvent(ShipBoard): void
+ notifyStashComponentEvent(ShipBoard): void
+ notifyValidateShipEvent(ShipBoard): void
+ notifyNewCardEvent(AdventureCard): void
+ notifyPeekForecastEvent(ShipBoard, int): void
+ notifyRequestFaceDownComponentEvent(ShipBoard, Component): void
+ notifyHourglassEndEvent(): void
+ notifyCabinInitializationEvent(ShipBoard, Point, CrewType): void
+ notifyRequestFaceUpComponentEvent(ShipBoard, Component): void
+ notifyShipPieceRemovalEvent(ShipBoard, int): void
+ notifyGrabStashedComponentEvent(ShipBoard, int): void
+ notifySurrenderRequestEvent(ShipBoard, SurrenderCause): void
+ notifyLoseCrewEvent(ShipBoard, Point): void
+ notifyGameEndEvent(Map<ShipBoard, Integer>): void
+ notifyPlaceComponentEvent(ShipBoard, Direction, Point): void
+ notifyShipNotConnectedEvent(ShipBoard, List<Set<Point>>): void
+ notifySurrenderEvent(List<ShipBoard>): void
+ notifyPlanetChoiceEvent(ShipBoard, int, ShipBoard): void
+ requestSnapshot(Game, ShipBoard): void
+ notifyGrabCreditsEvent(ShipBoard, int): void
+ notifyRejectComponentEvent(ShipBoard): void
+ notifyActivateComponentEvent(ShipBoard, Point, boolean): void
+ notifyUseBatteryEvent(ShipBoard, Point): void
+ notifyGameStateUpdateEvent(GameState): void
+ notifyGoodsUpdateEvent(ShipBoard, Point, GoodsType, boolean): void
+ notifyForecastDetailsEvent(ShipBoard, List<AdventureCard>): void
+ notifyReleaseForecastEvent(ShipBoard, int): void
+ notifyRemoveComponentEvent(ShipBoard, Point): void
+ notifyStartAdventure(): void
+ notifyGrabPlacedComponentEvent(ShipBoard): void
+ notifyFlightBoardUpdateEvent(ShipBoard, int): void
+ notifyFlipHourglassEvent(ShipBoard): void
- getGameSnapshot(Game): GameSnapshot

startAdventureCallback: Runnable

<<interface>>
GameFactory

+ createFlightBoard(int, GameEventListener): FlightBoard
+ createShipBoard(GameColor, GameEventListener): ShipBoard
+ createScoresRegistry(): ScoresRegistry
+ createShipBuildingState(): ShipBuildingState
+ createDeck(Game): Deck
+ createSurrenderPolicy(GameEventListener): SurrenderPolicy
+ createShipCorrectionState(): ShipCorrectionState
+ getFactory(Level): GameFactory

GameFactory

+ GameFactory():

+ createFlightBoard(): FlightBoard
+ getFactory(Level): GameFactory
+ createShipBoard(GameColor): ShipBoard
+ createShipBuildingState(): ShipBuildingState

<<interface>>
GameInterface

+ placeComponent(ShipBoard, Point, Direction): void
+ requestComponent(ShipBoard, int): void
+ grabPlacedComponent(ShipBoard): void
+ placeShipOnFlightBoard(ShipBoard, int): void
+ initializeCabin(ShipBoard, Point, CrewType): void
+ placeGoods(ShipBoard, Point, GoodsType): void
+ addShipBoard(GameColor): ShipBoard
+ activateComponent(ShipBoard, Point): void
+ drawCard(ShipBoard): void
+ requestSnapshot(ShipBoard): void
+ stashComponent(ShipBoard): void
+ requestRandComponent(ShipBoard): void
+ grabReward(ShipBoard): void
+ chooseShipPiece(ShipBoard, int): void
+ useBattery(ShipBoard, Point): void
+ goNext(ShipBoard): void
+ start(): void
+ skip(ShipBoard): void
+ removeComponent(ShipBoard, Point): void
+ skipBuilding(): void
+ acquireForecast(ShipBoard, int): void
+ removeGoods(ShipBoard, Point, GoodsType): void
+ releaseForecast(ShipBoard): void
+ rejectComponent(ShipBoard): void
+ loseCrew(ShipBoard, Point): void
+ loseGood(ShipBoard, Point): void
+ placeShipOnFlightBoard(ShipBoard): void
+ flipHourglass(ShipBoard): void
+ giveUp(ShipBoard): void
+ grabStashedComponent(ShipBoard, int): void
+ choosePlanet(ShipBoard, int): void

startAdventureCallback: Runnable

GameModel

+ GameModel():

+ createGame(Level, int, GameEventListener): GameInterface

<<interface>>
GameModelInterface

+ createGame(Level, int, GameEventListener): GameInterface

<<record>>
GameSnapshot

+ GameSnapshot(FlightBoardDTO, ComplexStateDTO, Map<String, Ship

+ currentCardId(): int
+ ships(): Map<String, ShipBoardDTO>
+ flightBoardDTO(): FlightBoardDTO
+ state(): ComplexStateDTO

<<record>>
GameSnapshotEvent

+ GameSnapshotEvent(String, LobbyDetailsDTO, GameSnapshot):

+ shouldResume(): boolean
+ lobbyDetails(): LobbyDetailsDTO
+ playerName(): String
+ gameSnapshot(): GameSnapshot

GameState

+ GameState():

# game: Game

+ spendBatteries(ShipBoard, Point): void
+ rejectComponent(ShipBoard): void
+ removeComponent(ShipBoard, Point): void
+ flipHourglass(ShipBoard): void
+ acquireForecast(ShipBoard, int): void
+ initializeCabin(ShipBoard, Point, CrewType): void
+ stashComponent(ShipBoard): void
+ grabStashedComponent(ShipBoard, int): void
+ chooseShipPiece(ShipBoard, int): void
+ removeGood(ShipBoard, Point, GoodsType): void
+ placeComponent(ShipBoard, Point, Direction): void
+ giveUp(ShipBoard): void
+ activateComponent(ShipBoard, Point): void
+ cancelSkip(ShipBoard): void
+ drawCard(ShipBoard): void
+ choosePlanet(ShipBoard, int): void
+ loseGood(ShipBoard, Point): void
+ releaseForecast(ShipBoard): void
+ addGood(ShipBoard, Point, GoodsType): void
+ grabPlacedComponent(ShipBoard): void
+ placeShipOnFlightBoard(ShipBoard): void
+ goNext(ShipBoard): void
+ placeShipOnFlightBoard(ShipBoard, int): void
+ grabReward(ShipBoard): void
+ loseCrew(ShipBoard, Point): void
+ skip(ShipBoard): void
+ requestRandComponent(ShipBoard): void
+ requestComponent(ShipBoard, int): void

game: Game

GameState

+ GameState():

# game: Game
# myShip: ShipBoard

+ notifyRequestRandComponent(ShipBoard, Component): void
+ notifySurrenderShip(Set<ShipBoard>): void
+ notifyRemoveComponent(ShipBoard, Point): void
+ notifyInitializeCabin(ShipBoard, Point, CrewType): void
+ notifyShipValidated(ShipBoard): void
+ notifyDrawCard(AdventureCard): void
+ notifyCurrentPlayerUpdate(ShipBoard): void
+ notifyFlightBoardPosition(ShipBoard, int, boolean): void
+ notifyPeekForecast(ShipBoard, int): void
+ notifySurrenderRequest(Player): void
+ notifyFlipHourglass(ShipBoard): void
+ notifyGrabReward(ShipBoard, int): void
+ notifyShipNotConnected(ShipBoard, List<Set<Point>>): void
+ notifyStashComponent(ShipBoard): void
+ notifyReleaseForecast(ShipBoard, boolean): void
+ notifyUseBattery(ShipBoard, Point): void
+ leave(): void
+ notifyRejectComponent(ShipBoard): void
+ notifyHourglassEnd(): void
+ notifyActivateComponent(ShipBoard, Point): void
+ notifyRemoveGoods(ShipBoard, Point, GoodsType): void
+ notifyChooseShipPiece(ShipBoard, int): void
+ notifyRequestComponent(ShipBoard, Component): void
+ notifyGrabPlacedComponent(ShipBoard): void
+ notifyChoosePlanet(ShipBoard, int, ShipBoard): void
+ notifyLoseCrew(ShipBoard, Point): void
+ notifyPlaceComponent(ShipBoard, Point, Direction): void
+ notifyPlaceGoods(ShipBoard, Point, GoodsType): void
+ notifyGrabStashedComponent(ShipBoard, int): void
+ notifyGiveUp(ShipBoard): void

availableActions: List<StateActions>
game: Game
myShip: ShipBoard
forecastDeck: List<AdventureCard>

<<interface>>
GameStateInterface

<<interface>>
GameStateInterface

<<record>>
GameStateUpdateEvent

+ GameStateUpdateEvent(StateDTO):

+ stateDTO(): StateDTO

GiveUp

+ GiveUp():

+ execute(VirtualServer): void

GoNext

+ GoNext():

+ execute(VirtualServer): void

GoodsBuffer

+ GoodsBuffer(Map<GoodsType, Integer>):

- goodsBuffer: Map<GoodsType, Integer>

+ addToBuffer(GoodsType): void
+ removeFromBuffer(GoodsType): void

goodsBuffer: Map<GoodsType, Integer>
empty: boolean

GoodsLoss

+ GoodsLoss(int):

GoodsLoss

+ GoodsLoss(int):

+ inflictPenalty(ShipBoard, FlightBoard): Optional<AdventureState>

<<enumeration>>

GoodsType

- GoodsType(int):

- value: int

+ valueOf(String): GoodsType
+ values(): GoodsType[]

<<record>>
GoodsUpdateEvent

+ GoodsUpdateEvent(String, Point, GoodsType, boolean):

+ goodsType(): GoodsType
+ playerName(): String
+ add(): boolean
+ point(): Point

<<record>>
GrabCreditsEvent

+ GrabCreditsEvent(String, int):

+ playerName(): String
+ value(): int

GrabPlacedComponent

+ GrabPlacedComponent():

+ execute(VirtualServer): void

<<record>>
GrabPlacedComponentEvent

+ GrabPlacedComponentEvent(String):

+ playerName(): String

GrabReward

+ GrabReward():

+ execute(VirtualServer): void

GrabRewardState

+ GrabRewardState(ShipBoard, ShipBoard):

+ notifyGrabReward(ShipBoard, int): void

availableActions: List<StateActions>

GrabRewardState

+ GrabRewardState(ShipBoard, Runnable):

- shipBoard: ShipBoard

+ skip(ShipBoard): void
+ grabReward(ShipBoard): void
+ goNext(ShipBoard): void

shipBoard: ShipBoard

GrabStashedComponent

+ GrabStashedComponent(int):

+ execute(VirtualServer): void

<<record>>
GrabStashedComponentEvent

+ GrabStashedComponentEvent(String, int):

+ playerName(): String
+ index(): int

GuiActivationScreen

+ GuiActivationScreen(ClientModel, ClientControllerInterface, Adve

+ notifyActivateComponent(ShipBoard, Point): void
+ notifyUseBattery(ShipBoard, Point): void

guiController: GuiController

GuiAdventureCard

+ GuiAdventureCard(int):

GuiAdventureCardRegistry

- GuiAdventureCardRegistry():

- instance: GuiAdventureCardRegistry

instance: GuiAdventureCardRegistry

GuiAdventureScreen

+ GuiAdventureScreen(ClientModel, ClientControllerInterface, Adven

# guiAdventureCard(): Optional<GuiAdventureCard>
# getShipBoardVBox(ShipBoard): VBox

myTurn: boolean
freeUseVBox: VBox

GuiBattery

+ GuiBattery(Battery):

+ notifyContentChange(): void

GuiCabin

+ GuiCabin(Cabin):

+ notifyContentChange(): void

GuiCargoHold

+ GuiCargoHold(CargoHold):

+ notifyContentChange(): void

GuiComponent

# GuiComponent(Component):

+ hasId(int): boolean
+ notifyContentChange(): void
+ of(Component): GuiComponent
+ clearHighlight(): void
# updateContentBox(List<Color>): void

highlight: Color
direction: Direction

GuiComponentBank

+ GuiComponentBank(ComponentBank, GuiController):

+ notifyRequestComponent(Component): void
+ notifyRejectComponent(Component): void
+ notifyRequestRandComponent(): void

GuiComponentRegistry

- GuiComponentRegistry():

- instance: GuiComponentRegistry

instance: GuiComponentRegistry

GuiController

+ GuiController():

+ releaseForecast(): void
+ drawCard(): void
+ requestComponent(int): void
+ requestRandComponent(): void
+ placeShipOnFlightBoard(int): void
+ rejectComponent(): void
+ acquireForecast(int): void
+ rotateHandComponent(): void
+ goNext(): void
+ choosePlanet(int): void
+ grabStashedComponent(int): void
+ handlePointPress(Point): void
+ stashComponent(): void

GuiCorrectionScreen

+ GuiCorrectionScreen(ClientModel, ClientControllerInterface, ShipC

+ notifyShipNotConnected(ShipBoard, List<Set<Point>>): void
+ notifyShipValidated(ShipBoard): void
- clearSelection(): void
+ notifyChooseShipPiece(ShipBoard, int, List<Point>): void
- showConfirmationButton(String): void
- confirmSelection(): void
- shipNotConnected(ShipBoard, List<Set<Point>>): void
# getShipBoardVBox(ShipBoard): VBox
+ notifyRemoveComponent(ShipBoard, Point): void

freeUseVBox: VBox
guiController: GuiController

GuiCrewInitializationScreen

+ GuiCrewInitializationScreen(ClientModel, ClientControllerInterfa

# getShipBoardVBox(ShipBoard): VBox
+ notifyInitializeCabin(ShipBoard, Point, CrewType, int): void

freeUseVBox: VBox
guiController: GuiController

GuiDeclareEnginePowerScreen

+ GuiDeclareEnginePowerScreen(ClientModel, ClientControllerInter

GuiDeclareFirePowerScreen

+ GuiDeclareFirePowerScreen(ClientModel, ClientControllerInterfac

GuiEndGameScreen

+ GuiEndGameScreen(ClientModel, ClientControllerInterface):

- createScoreRow(String, Integer): HBox

node: Pane

GuiFlightBoard

+ GuiFlightBoard(FlightBoard, GuiController):

+ notifyFlightBoardPosition(ShipBoard, int): void
- updateFlightBoardHBox(): void

GuiForecast

+ GuiForecast(ShipBoard[], GuiController):

- createTakenForecast(ShipBoard): Rectangle
- updateCoveredDecks(): void
- createCardBase(Color): Rectangle
- createFreeForecast(): Rectangle
+ notifyOtherReleaseForecast(ShipBoard, int): void
+ notifyOtherPeekForecast(ShipBoard, int): void
+ notifyMeReleaseForecast(): void
+ notifyMePeekForecast(): void

forecastDeck: List<AdventureCard>

GuiGameCreationScreen

+ GuiGameCreationScreen(ClientModel, ClientControllerInterface):

node: Pane

GuiGameScreen

+ GuiGameScreen(ClientModel, ClientControllerInterface, GameState)

+ notifyComponentChange(ShipBoard, Point): void
# getShipBoardVBox(ShipBoard): VBox
+ notifyRemoveComponent(ShipBoard, Point): void

node: Pane
freeUseVBox: VBox
centralVBox: VBox
guiController: GuiController
otherShipsVBox: PurpleVBox
bottomBox: HBox
sideVBox: VBox

GuiGoodsScreen

+ GuiGoodsScreen(ClientModel, ClientControllerInterface, AddGoodsS

- updateBufferBox(): void
+ notifyComponentChange(ShipBoard, Point): void

freeUseVBox: VBox
guiController: GuiController
buttonsBox: HBox

GuiHand

+ GuiHand(Component, GuiController):

+ notifySetHand(Component): void
- createPlaceholder(): Region
+ notifyClearHand(): void

componentDirection: Direction

<<enumeration>>
GuiHighlights

- GuiHighlights(Color):

- color: Color

+ valueOf(String): GuiHighlights
+ getSomeColors(int): List<GuiHighlights>
+ values(): GuiHighlights[]

GuiImageRegistry

+ GuiImageRegistry():

+ getImage(int): Image
# loadImages(Map<Integer, Path>): void

GuiJoinOrCreateScreen

+ GuiJoinOrCreateScreen(ClientModel, Set<UUID>, ClientControllerI

- createLobbyRow(Lobby, UUID): HBox
+ notifyNewLobby(Lobby): void
+ notifyRemoveLobby(UUID): void
- updateLobbyButtons(): void

node: Pane

GuiLobbyScreen

+ GuiLobbyScreen(ClientModel, ClientControllerInterface):

+ notifyPlayerJoin(Player): void
- createPlayerBox(String): StackPane

node: Pane

GuiLog

+ GuiLog():

+ log(String): void

GuiLoseGoodsScreen

+ GuiLoseGoodsScreen(ClientModel, ClientControllerInterface, Remov

- countCargoHoldsWithGoods(): int
- setupButtonBox(): void
- updateCargoInfoBox(): void
+ notifyUseBattery(ShipBoard, Point): void
+ notifyComponentChange(ShipBoard, Point): void
- countBatteries(): int

freeUseVBox: VBox
guiController: GuiController

GuiNewCardScreen

+ GuiNewCardScreen(ClientModel, ClientControllerInterface, DrawCar

+ notifyDrawCard(AdventureCard): void

guiController: GuiController

GuiNicknameChoiceScreen

+ GuiNicknameChoiceScreen(ClientModel, ClientControllerInterface):

- submitNickname(TextField): void

node: Pane

GuiPlanetScreen

+ GuiPlanetScreen(ClientModel, ClientControllerInterface, ChooseP

+ notifyChoosePlanet(ShipBoard, int, ShipBoard): void
+ notifyCurrentPlayerUpdate(ShipBoard): void
- createPlanetButton(int): CircularToggleButton

guiController: GuiController

GuiProjectilesScreen

+ GuiProjectilesScreen(ClientModel, ClientControllerInterface, Ha

projectileVBox: PurpleVBox
freeUseVBox: VBox

GuiRemoveCrewScreen

+ GuiRemoveCrewScreen(ClientModel, ClientControllerInterface, Remo

+ notifyComponentChange(ShipBoard, Point): void
- countCabinsWithCrew(): int
- setupButtonBox(): void
- updateCabinInfoBox(): void

freeUseVBox: VBox
guiController: GuiController

GuiRewardScreen

+ GuiRewardScreen(ClientModel, ClientControllerInterface, GrabRew

- setupButtons(): void
- updateLog(): void

freeUseVBox: VBox
guiController: GuiController
buttonsBox: HBox

GuiScreen

+ GuiScreen(ClientModel, ClientControllerInterface):
+ GuiScreen(ClientModel, ClientControllerInterface, GameState):

node: Pane

GuiScreenFactory

+ GuiScreenFactory():

+ createScreen(GameState, ClientModel, ClientControllerInterface): GuiScreen
+ createScreen(MetaState, ClientModel, ClientControllerInterface): GuiScreen

GuiSecondShipBuildingScreen

+ GuiSecondShipBuildingScreen(ClientModel, ClientControllerInterface

+ notifyStashComponent(ShipBoard, Component): void
+ notifyReleaseForecast(ShipBoard, int): void
- updateHourglassButton(): void
+ notifyFlipHourglass(ShipBoard): void
- setupHourglassButton(): void
+ notifyPeekForecast(ShipBoard, int): void
+ notifyStashComponent(ShipBoard, Component, Point): void
# getShipBoardVBox(ShipBoard): VBox
+ notifyHourglassEnd(): void
+ notifyGrabStashedComponent(ShipBoard, int, Component): void

forecastDeck: List<AdventureCard>
freeUseVBox: VBox
guiController: GuiController

GuiShipBoard

+ GuiShipBoard(String, ShipBoard, GuiController):

- shipGrid: GridPane

- getEmptyAreaPane(Point): StackPane
+ notifyPlaceComponent(Component, Point, Direction): void
+ clearHighlights(): void
+ notifyRemoveComponent(Point): void
+ notifyComponentChange(Point): void
+ highlightPoints(Set<Point>, Color): void

shipGrid: ShipBoard

GuiShipBuildingScreen

+ GuiShipBuildingScreen(ClientModel, ClientControllerInterface, Ship

+ notifyRejectComponent(ShipBoard, Component, Point): void
+ notifyFlightBoardPosition(ShipBoard, int): void
+ notifyPlaceComponent(ShipBoard, Point, Direction): void
+ notifyRejectComponent(ShipBoard, Component): void
+ notifyRequestComponent(ShipBoard, Component): void
+ notifyPlaceComponent(ShipBoard, Point, Direction, Point): void
+ notifyRequestRandComponent(ShipBoard, Component): void

GuiShipPieceChoiceScreen

+ GuiShipPieceChoiceScreen(ClientModel, ClientControllerInterface

- setupButtonBox(): void
- highlightShipPieces(): void

guiController: GuiController

GuiStash

+ GuiStash(List<Component>, GuiController):

+ notifyStash(Component): void
+ notifyGrab(int): void

GuiStatBox

+ GuiStatBox(ShipBoard):

- buildGrid(): void
- createLabel(String, double): Label
- setupContainer(): void
+ notifyChange(): void

GuiTestShipBuildingScreen

+ GuiTestShipBuildingScreen(ClientModel, ClientControllerInterfac

# getShipBoardVBox(ShipBoard): VBox

freeUseVBox: VBox
guiController: GuiController

GuiView

+ GuiView(ClientController, ClientModel):

+ notifyRejectComponent(ShipBoard, Component): void
+ notifyGiveUp(Player): void
+ notifyCurrentState(GameState): void
+ notifyStashComponent(ShipBoard, Component): void
+ notifyActivateComponent(ShipBoard, Point): void
+ notifyDrawCard(AdventureCard): void
+ notifyFlightBoardPosition(ShipBoard, int): void
+ notifyRemoveComponent(ShipBoard, Point): void
+ notifyChoosePlanet(ShipBoard, int, ShipBoard): void
+ notifyHourglassEnd(): void
+ notifyPeekForecast(ShipBoard, int): void
+ notifyRejectComponent(ShipBoard, Component, Point): void
+ notifyPlaceComponent(ShipBoard, Point, Direction, Point): void
+ notifyRequestRandComponent(ShipBoard, Component): void
+ notifyRequestComponent(ShipBoard, Component): void
+ notifyChooseShipPiece(ShipBoard, int, List<Point>): void
+ notifyPlaceComponent(ShipBoard, Point, Direction): void
+ notifyCurrentPlayerUpdate(ShipBoard): void
+ notifyNewLobby(Lobby): void
+ notifyShipNotConnected(ShipBoard, List<Set<Point>>): void
+ notifyStashComponent(ShipBoard, Component, Point): void
+ notifyInitializeCabin(ShipBoard, Point, CrewType, int): void
+ notifyGrabStashedComponent(ShipBoard, int, Component): void
- createRootWithBackground(): StackPane
+ notifyGrabPlacedComponent(ShipBoard, Point): void
+ notifyShipValidated(ShipBoard): void
+ notifyPlayerJoin(Player): void
+ notifyGrabReward(ShipBoard, boolean): void
- switchToScreen(GuiScreen): void
+ notifyPlaceGoods(ShipBoard, Point, GoodsType): void
+ notifyFlipHourglass(ShipBoard): void
+ notifyLoseCrew(ShipBoard, Point): void
+ notifyUseBattery(ShipBoard, Point): void
+ notifyMetaState(MetaState): void
+ notifyRemoveGoods(ShipBoard, Point, GoodsType): void
+ reportError(String): void
+ notifyReleaseForecast(ShipBoard, int): void
+ notifyRemoveLobby(UUID): void

forecastDeck: List<AdventureCard>
stage: Stage
finalScores: Map<Player, Integer>

<<record>>
HandleProjectileDTO

+ HandleProjectileDTO(String, ProjectileType, int, Direction, Set<P

+ direction(): Direction
+ playerName(): String
+ diceRoll(): int
+ availablePoints(): Set<Point>
+ projectileType(): ProjectileType

HandleProjectileState

+ HandleProjectileState(ShipBoard, Projectile):

- projectile: Projectile

+ activateComponent(ShipBoard, Point): void
+ spendBatteries(ShipBoard, Point): void
# endStateAction(): void

projectile: Projectile

HandleProjectileState

+ HandleProjectileState(ShipBoard, ShipBoard, Projectile, Set<Poin

- projectile: Projectile

+ notifyRemoveComponent(ShipBoard, Point): void

projectile: Projectile

Hourglass

+ Hourglass(int):

- flipsLeft: int
- timeLeft: AtomicLong
- isRunning: AtomicBoolean

+ setup(int, int, boolean): void
+ end(): void
- start(): void
+ flip(): void

timeLeft: long
isRunning: boolean
flipsLeft: int

Hourglass

+ Hourglass(int):

- duration: int
- missingTime: int
- flipsLeft: int

+ stop(): void
+ flip(Runnable): void

lastFlip: boolean
isRunning: boolean
flipsLeft: int
missingTime: int
duration: int

<<record>>
HourglassDTO

+ HourglassDTO(int, int, boolean):

- isRunning: boolean

+ flipsLeft(): int
+ timeLeft(): int

<<record>>
HourglassEndEvent

+ HourglassEndEvent():

InitializeCabin

+ InitializeCabin(Point, CrewType):

+ execute(VirtualServer): void

<<record>>
InitializeCabinEvent

+ InitializeCabinEvent(String, Point, CrewType):

+ playerName(): String
+ crewType(): CrewType
+ i t() P i t

JFXApp

+ JFXApp():

- guiView: GuiView

+ start(Stage): void

guiView: GuiView

JoinLobby

+ JoinLobby(UUID):

+ execute(VirtualServer): void

<<record>>
JoinLobbyEvent

+ JoinLobbyEvent(String, GameColor):

+ playerName(): String
+ color(): GameColor

JsonUtils

+ JsonUtils():

+ nodeToConnector(JsonNode): Map<Direction, Connector>
+ serializeShipBoardDTO(ShipBoardDTO): JsonNode
+ deserializeShipBoardDTO(JsonNode): ShipBoardDTO

LeaveLobby

+ LeaveLobby():

+ execute(VirtualServer): void

<<enumeration>>
Level

+ Level():

+ values(): Level[]
+ valueOf(String): Level

LifeSupport

+ LifeSupport(Map<Direction, Connector>, int, CrewType):

alienType: CrewType

LifeSupport

+ LifeSupport(Map<Direction, Connector>, int, CrewType):
+ LifeSupport(Map<Direction, Connector>, CrewType):
+ LifeSupport(CrewType):

- checkCrewType(): void

alienType: CrewType

Lobby

+ Lobby(UUID, int, Level, List<String>, String):
+ Lobby(UUID, int, Level, String):

- playersN: int
- level: Level
- players: List<String>
- host: String
- id: UUID

level: Level
host: String
id: UUID
players: List<String>
playersN: int

Lobby

+ Lobby(boolean, boolean, GameModelInterface, Player, Level, int, Cons
+ Lobby(GameModelInterface, Player, Level, int, Consumer<Lobby>):

- state: LobbyState
- id: UUID
- playerColors: Map<Player, GameColor>
- level: Level
- eventQueue: EventQueue<LobbyEvent>
- host: Player
- removalDelay: long
- players: List<Player>
- numPlayers: int

+ placeShipOnFlightBoard(Player): void
+ notifyPlayerReconnection(Player): void
+ removeGoods(Player, Point, GoodsType): void
+ requestComponent(Player, int): void
+ chooseShipPiece(Player, int): void
- checkInGame(): void
+ grabPlacedComponent(Player): void
- startGame(): void
+ stopEventHandler(): void
+ requestRandComponent(Player): void
- shouldSchedule(): boolean
+ placeShipOnFlightBoard(Player, int): void
+ addPlayer(Player): boolean
+ remove(): void
+ removeComponent(Player, Point): void
+ goNext(Player): void
+ flipHourglass(Player): void
+ choosePlanet(Player, int): void
+ placeGoods(Player, Point, GoodsType): void
- checkLobbyState(LobbyState): void
+ acquireForecast(Player, int): void
+ skip(Player): void
+ drawCard(Player): void
- updatePending(): void
+ stashComponent(Player): void
+ initializeCabin(Player, Point, CrewType): void
+ releaseForecast(Player): void
+ notifyPlayerExit(Player): void
# saveShips(): void
+ rejectComponent(Player): void
+ grabStashedComponent(Player, int): void
+ loseCrew(Player, Point): void
+ grabReward(Player): void
+ useBattery(Player, Point): void
+ activateComponent(Player, Point): void
+ giveUp(Player): void
- loadScenario(): void
+ loseGoods(Player, Point): void
+ notifyPlayerDisconnection(Player): void
+ placeComponent(Player, Point, Direction): void

playerColors: Map<Player, GameColor>
level: Level
state: LobbyState
removalDelay: long
host: Player
numPlayers: int
id: UUID
eventQueue: EventQueue<LobbyEvent>
players: List<Player>
scenarioPath: String

<<record>>
LobbyDetailsDTO

+ LobbyDetailsDTO(UUID, Map<String, GameColor>, Level, int):

+ lobbyId(): UUID
+ playersN(): int
+ playerColors(): Map<String, GameColor>
+ level(): Level

<<record>>
LobbyDetailsEvent

+ LobbyDetailsEvent(String, LobbyDetailsDTO):

+ playerName(): String
+ details(): LobbyDetailsDTO

<<interface>>
LobbyEvent

+ runLobbyAction(Lobby): void

LobbyEventHandler

+ LobbyEventHandler(EventQueue<LobbyEvent>, Lobby):

- sendEvent(String, LobbyEvent): void
+ handleEvent(LobbyEvent): void
- broadcastEvent(LobbyEvent): void

players: List<Player>

<<interface>>
LobbyInterface

+ flipHourglass(Player): void
+ placeComponent(Player, Point, Direction): void
+ requestComponent(Player, int): void
+ chooseShipPiece(Player, int): void
+ removeGoods(Player, Point, GoodsType): void
+ useBattery(Player, Point): void
+ placeShipOnFlightBoard(Player): void
+ requestRandComponent(Player): void
+ placeGoods(Player, Point, GoodsType): void
+ goNext(Player): void
+ giveUp(Player): void
+ grabStashedComponent(Player, int): void
+ initializeCabin(Player, Point, CrewType): void
+ grabReward(Player): void
+ activateComponent(Player, Point): void
+ stashComponent(Player): void
+ removeComponent(Player, Point): void
+ loseGoods(Player, Point): void
+ acquireForecast(Player, int): void
+ grabPlacedComponent(Player): void
+ releaseForecast(Player): void
+ drawCard(Player): void
+ choosePlanet(Player, int): void
+ placeShipOnFlightBoard(Player, int): void
+ loseCrew(Player, Point): void
+ rejectComponent(Player): void

<<enumeration>>
LobbyState

+ LobbyState():

+ values(): LobbyState[]
+ valueOf(String): LobbyState

LockUtils

+ LockUtils():

+ withLock(ReadWriteLock, Runnable, boolean): void
+ withLock(ReadWriteLock, Supplier<T>, boolean): T

Logger

+ Logger():

+ println(int, Object[]): void

enable: boolean

LoseCrew

+ LoseCrew(Point):

+ execute(VirtualServer): void

<<record>>
LoseCrewEvent

+ LoseCrewEvent(String, Point):

+ point(): Point
+ playerName(): String

LoseGoods

+ LoseGoods(Point):

+ execute(VirtualServer): void

Main

+ Main():

- startClient(): void
+ main(String[]): void
- startServer(): void

<<interface>>
Message

<<enumeration>>
MetaState

+ MetaState():

+ valueOf(String): MetaState
+ values(): MetaState[]

MeteorSwarmCard

+ MeteorSwarmCard(Game, Level, List<Projectile>, int):

+ initialize(): void

currentShipBoard: ShipBoard
nextState: AdventureState

MeteorSwarmCard

+ MeteorSwarmCard(Level, List<Projectile>, int):

- projectiles: List<Projectile>

projectiles: List<Projectile>

<<interface>>
ModelObserver

+ notifyRemoveLobby(UUID): void
+ notifyStashComponent(ShipBoard, Component, Point): void
+ notifyRequestComponent(ShipBoard, Component): void
+ notifyFlightBoardPosition(ShipBoard, int): void
+ notifyCurrentPlayerUpdate(ShipBoard): void
+ notifyShipNotConnected(ShipBoard, List<Set<Point>>): void
+ notifyGrabPlacedComponent(ShipBoard, Point): void
+ notifyMetaState(MetaState): void
+ notifyGrabStashedComponent(ShipBoard, int, Component): void
+ notifyPlaceComponent(ShipBoard, Point, Direction, Point): void
+ notifyChoosePlanet(ShipBoard, int, ShipBoard): void
+ notifyRemoveComponent(ShipBoard, Point): void
+ notifyActivateComponent(ShipBoard, Point): void
+ notifyPlayerJoin(Player): void
+ notifyPlaceComponent(ShipBoard, Point, Direction): void
+ notifyReleaseForecast(ShipBoard, int): void
+ notifyUseBattery(ShipBoard, Point): void
+ notifyStashComponent(ShipBoard, Component): void
+ notifyShipValidated(ShipBoard): void
+ notifyRejectComponent(ShipBoard, Component): void
+ notifyHourglassEnd(): void
+ notifyRemoveGoods(ShipBoard, Point, GoodsType): void
+ notifyInitializeCabin(ShipBoard, Point, CrewType, int): void
+ notifyGiveUp(Player): void
+ notifyFlipHourglass(ShipBoard): void
+ notifyDrawCard(AdventureCard): void
+ notifyLoseCrew(ShipBoard, Point): void
+ notifyPeekForecast(ShipBoard, int): void
+ notifyChooseShipPiece(ShipBoard, int, List<Point>): void
+ notifyGrabReward(ShipBoard, boolean): void
+ notifyRejectComponent(ShipBoard, Component, Point): void
+ notifyNewLobby(Lobby): void
+ notifyPlaceGoods(ShipBoard, Point, GoodsType): void
+ notifyCurrentState(GameState): void
+ notifyRequestRandComponent(ShipBoard, Component): void

forecastDeck: List<AdventureCard>
finalScores: Map<Player, Integer>

<<record>>
NewCardEvent

+ NewCardEvent(int):

+ cardId(): int

NoSurrenderPolicy

+ NoSurrenderPolicy():

+ confirmSurrender(FlightBoard): Set<ShipBoard>
+ requestSurrender(ShipBoard, SurrenderCause): boolean

surrenderedShips: Set<ShipBoard>
surrenderEnabled: boolean

OpenSpaceCard

+ OpenSpaceCard(Game, Level, int):

nextState: AdventureState

OpenSpaceCard

+ OpenSpaceCard(Level, int):

<<record>>
PeekForecastEvent

+ PeekForecastEvent(String, int):

+ playerName(): String
+ forecastIndex(): int

<<interface>>
Penalty

+ inflictPenalty(ShipBoard, FlightBoard): Optional<AdventureState>

<<interface>>
Penalty

<<record>>
Ping

+ Ping():
+ Ping(UUID):

+ id(): UUID

PiratesCard

+ PiratesCard(Game, Level, int, int, int, List<Projectile>, int):

- projectiles: List<Projectile>
- firePowerThreshold: int
- creditPrize: int
- defeatedPlayers: List<ShipBoard>
- currentProjectile: Projectile
- defeated: boolean
- flightDaysLoss: int
- winnerShipBoard: ShipBoard

+ getReward(): void
+ initialize(): void

defeated: boolean
winnerShipBoard: ShipBoard
creditPrize: int
firePowerThreshold: int
projectiles: List<Projectile>
defeatedPlayers: List<ShipBoard>
flightDaysLoss: int
nextState: AdventureState
currentProjectile: Projectile

PiratesCard

+ PiratesCard(Level, int, int, int, List<Projectile>, int):

- firePowerThreshold: int
- creditPrize: int
- flightDaysLoss: int
- projectiles: List<Projectile>

creditPrize: int
firePowerThreshold: int
projectiles: List<Projectile>
flightDaysLoss: int

PlaceComponent

+ PlaceComponent(Point, Direction):

+ execute(VirtualServer): void

<<record>>
PlaceComponentEvent

+ PlaceComponentEvent(String, Point, Direction):

+ playerName(): String
+ position(): Point
+ t ti () Di ti

PlaceGoods

+ PlaceGoods(Point, GoodsType):

+ execute(VirtualServer): void

PlaceShipOnFlightBoard

+ PlaceShipOnFlightBoard(int):
+ PlaceShipOnFlightBoard():

+ execute(VirtualServer): void

<<record>>
PlanetChoiceEvent

+ PlanetChoiceEvent(String, int, String):

+ planetIndex(): int
+ nextPlayerName(): String
+ l N () St i

PlanetsCard

+ PlanetsCard(Game, Level, List<Map<GoodsType, Integer>>, int, int)

- planetChoices: Map<ShipBoard, Integer>
- landedShips: List<ShipBoard>
- flightDaysLoss: int
- currentShipBoard: ShipBoard
- planetChoiceAllowed: boolean

~ choosePlanet(ShipBoard, int): void
+ initialize(): void

planetChoiceAllowed: boolean
landedShips: List<ShipBoard>
currentShipBoard: ShipBoard
planetChoices: Map<ShipBoard, Integer>
flightDaysLoss: int
nextState: AdventureState

PlanetsCard

+ PlanetsCard(Level, List<Map<GoodsType, Integer>>, int, int):

- planets: List<Map<GoodsType, Integer>>
- flightDaysLoss: int

flightDaysLoss: int
planets: List<Map<GoodsType, Integer>>

Player

- Player(String):

- nickname: String
- shipBoard: AtomicReference<ShipBoard>
- lobby: AtomicReference<Lobby>

+ getPlayer(ShipBoard): Player
+ getPlayer(String): Player
+ hashCode(): int
+ equals(Object): boolean
+ removePlayer(String): void
+ addPlayer(String): Player
+ clear(): void
+ leaveLobby(): void

shipBoard: Optional<ShipBoard>
lobby: Optional<Lobby>
allPlayers: Set<Player>
nickname: String

Player

+ Player(String):

- nickname: String
- shipBoard: ShipBoard

shipBoard: ShipBoard
nickname: String

<<record>>
PlayerDisconnectionEvent

+ PlayerDisconnectionEvent(String):

+ playerName(): String

<<record>>
PlayerExitEvent

+ PlayerExitEvent(String):

+ playerName(): String
+ runLobbyAction(Lobby): void

PlayerRegistry

+ PlayerRegistry():

+ removePlayer(Player): void
+ clear(): void
+ addPlayer(String): Player
+ getByNickname(String): Player

PointKeyDeserializer

+ PointKeyDeserializer():

+ deserializeKey(String, DeserializationContext): Object

PointSerializer

+ PointSerializer():

+ serialize(Point, JsonGenerator, SerializerProvider): void

<<record>>
Projectile

+ Projectile(int, Direction, ProjectileType):

+ roll(): int
+ type(): ProjectileType
+ di ti () Di ti

Projectile

+ Projectile(IntSupplier, Direction):
+ Projectile(Direction):

# direction: Direction
# diceRoll: int

+ getActivatablePoints(ShipBoard): Set<Point>
+ fireAt(ShipBoard): boolean
# getFirstFoundComponentPosition(ShipBoard): Optional<Point>
# getComponentPositionToRemove(ShipBoard): Optional<Point>

projectileType: ProjectileType
diceRoll: int
direction: Direction

ProjectileThreat

+ ProjectileThreat(List<Projectile>):

+ inflictPenalty(ShipBoard, FlightBoard): Optional<AdventureState>

ProjectileThreat

+ ProjectileThreat(List<Projectile>):

<<enumeration>>
ProjectileType

+ ProjectileType():

+ values(): ProjectileType[]
+ valueOf(String): ProjectileType

PurpleHBox

+ PurpleHBox():
+ PurpleHBox(double):

- setPurpleStyle(): void

PurpleVBox

+ PurpleVBox(double):
+ PurpleVBox(Node):
+ PurpleVBox():

- setPurpleStyle(): void

RegisterNickname

+ RegisterNickname(String):

+ execute(VirtualServer): void

RegisteredRequest

+ RegisteredRequest():

RejectComponent

+ RejectComponent():

+ execute(VirtualServer): void

<<record>>
RejectComponentEvent

+ RejectComponentEvent(String):

+ playerName(): String

ReleaseForecast

+ ReleaseForecast():

+ execute(VirtualServer): void

<<record>>
ReleaseForecastEvent

+ ReleaseForecastEvent(String, int):

+ deckIndex(): int
+ playerName(): String

<<interface>>
RemoteClient

+ notifyEvent(Event): void

<<interface>>
RemoteController

+ leaveLobby(): void
+ choosePlanet(int): void
+ giveUp(): void
+ grabReward(): void
+ loseCrew(Point): void
+ goNext(): void
+ rejectComponent(): void
+ flipHourglass(): void
+ joinLobby(UUID): void
+ stashComponent(): void
+ removeComponent(Point): void
+ ping(): void
+ placeShipOnFlightBoard(): void
+ drawCard(): void
+ useBattery(Point): void
+ newGame(Level, int): void
+ loseGoods(Point): void
+ initializeCabin(Point, CrewType): void
+ requestComponent(int): void
+ acquireForecast(int): void
+ requestRandComponent(): void
+ placeShipOnFlightBoard(int): void
+ grabStashedComponent(int): void
+ chooseShipPiece(int): void
+ placeGoods(Point, GoodsType): void
+ releaseForecast(): void
+ removeGoods(Point, GoodsType): void
+ activateComponent(Point): void
+ grabPlacedComponent(): void
+ placeComponent(Point, Direction): void

<<interface>>
RemoteRunnable

+ run(): void

<<interface>>
RemoteServer

+ registerNickname(RemoteClient, String): RemoteController

<<record>>
RemoveActiveLobbyEvent

+ RemoveActiveLobbyEvent(UUID):

+ lobbyId(): UUID

RemoveComponent

+ RemoveComponent(Point):

+ execute(VirtualServer): void

<<record>>
RemoveComponentEvent

+ RemoveComponentEvent(String, Point):

+ point(): Point
+ playerName(): String

<<record>>
RemoveCrewDTO

+ RemoveCrewDTO(String, int):

+ playerName(): String
+ crewLoss(): int

RemoveCrewState

+ RemoveCrewState(ShipBoard, int, ShipBoard):

+ notifyLoseCrew(ShipBoard, Point): void

availableActions: List<StateActions>

RemoveCrewState

+ RemoveCrewState(int, ShipBoard):

- crewSacrifice: int
- shipBoard: ShipBoard

+ skip(ShipBoard): void
+ loseCrew(ShipBoard, Point): void

crewSacrifice: int
game: Game
shipBoard: ShipBoard

RemoveGoods

+ RemoveGoods(Point, GoodsType):

+ execute(VirtualServer): void

<<record>>
RemoveGoodsDTO

+ RemoveGoodsDTO(String, int):

+ playerName(): String
+ goodsLoss(): int

RemoveGoodsState

+ RemoveGoodsState(ShipBoard, int, ShipBoard):

- goodsToLose: int

+ notifyUseBattery(ShipBoard, Point): void
+ notifyRemoveGoods(ShipBoard, Point, GoodsType): void

availableActions: List<StateActions>
goodsToLose: int

RemoveGoodsState

+ RemoveGoodsState(int, ShipBoard):

- shipBoard: ShipBoard
- goodsToLose: int

+ loseGood(ShipBoard, Point): void
+ skip(ShipBoard): void
- computeMostValuableGood(): void
- tryStateTransition(): void

goodsToLose: int
game: Game
shipBoard: ShipBoard

Request

# Request():

- uuid: UUID

+ execute(VirtualServer): void

uuid: UUID

RequestComponent

+ RequestComponent(int):

+ execute(VirtualServer): void

<<record>>
RequestFaceDownComponentEvent

+ RequestFaceDownComponentEvent(String, int):

+ playerName(): String
+ componentId(): int

<<record>>
RequestFaceUpComponentEvent

+ RequestFaceUpComponentEvent(String, int):

+ playerName(): String
+ componentId(): int

RequestNewGame

+ RequestNewGame(Level, int):

+ execute(VirtualServer): void

RequestRandComponent

+ RequestRandComponent():

+ execute(VirtualServer): void

Response

+ Response(UUID, RuntimeException):
+ Response(UUID):

- uuid: UUID
- error: RuntimeException

+ isError(): boolean

error: RuntimeException
uuid: UUID

RmiClient

+ RmiClient():

- clientController: ServerToClientInterface

+ removeComponent(Point): void
+ useBattery(Point): void
+ requestComponent(int): void
+ grabReward(): void
+ activateComponent(Point): void
+ goNext(): void
+ flipHourglass(): void
+ choosePlanet(int): void
+ leaveLobby(): void
+ placeShipOnFlightBoard(int): void
+ drawCard(): void
+ placeShipOnFlightBoard(): void
+ requestRandComponent(): void
+ loseCrew(Point): void
+ notifyEvent(Event): void
+ stashComponent(): void
+ placeComponent(Point, Direction): void
+ chooseShipPiece(int): void
+ giveUp(): void
- connect(): void
+ registerNickname(String): void
+ grabPlacedComponent(): void
+ start(String, String, int): void
- handleNetworkError(RemoteException): void
+ releaseForecast(): void
+ placeGoods(Point, GoodsType): void
+ dropConnection(): void
- ping(): void
+ loseGoods(Point): void
+ reconnect(): boolean
+ initializeCabin(Point, CrewType): void
+ requestNewGame(Level, int): void
+ removeGoods(Point, GoodsType): void
+ grabStashedComponent(int): void
+ joinLobby(UUID): void
- runRemoteMethod(RemoteRunnable): void
+ rejectComponent(): void
+ acquireForecast(int): void

clientController: ServerToClientInterface

RmiClientHandler

+ RmiClientHandler(RemoteClient, ServerControllerInterface):

- running: boolean
- afterEach: Runnable
- player: Player

+ chooseShipPiece(int): void
+ placeShipOnFlightBoard(): void
+ placeComponent(Point, Direction): void
+ activateComponent(Point): void
+ useBattery(Point): void
+ placeShipOnFlightBoard(int): void
+ grabReward(): void
+ stashComponent(): void
+ flipHourglass(): void
+ removeGoods(Point, GoodsType): void
+ goNext(): void
+ start(): void
+ pauseEvents(): void
- runUpdateThread(): void
+ grabStashedComponent(int): void
+ loseGoods(Point): void
+ rejectComponent(): void
+ placeGoods(Point, GoodsType): void
+ giveUp(): void
+ loseCrew(Point): void
+ acquireForecast(int): void
+ requestRandComponent(): void
+ grabPlacedComponent(): void
+ notifyEvent(Event): void
+ initializeCabin(Point, CrewType): void
+ newGame(Level, int): void
- handleNetworkError(RemoteException): void
+ choosePlanet(int): void
+ leaveLobby(): void
+ releaseForecast(): void
+ removeComponent(Point): void
+ drawCard(): void
+ stop(): void
+ joinLobby(UUID): void
+ ping(): void
+ requestComponent(int): void

player: Player
afterEach: Runnable
lobby: LobbyInterface
running: boolean

RmiServer

+ RmiServer(ServerControllerInterface):

+ registerNickname(RemoteClient, String): RemoteController
+ start(String, int): void
+ stop(): void

SabotageCard

+ SabotageCard(Level, int):

SabotageCard

# SabotageCard(Game, Level, int):

nextState: AdventureState

SafeSocket

+ SafeSocket(Socket):

+ close(): void
+ read(): Message
+ write(Message): void

ScoresRegistry

+ ScoresRegistry(int[], int):

- BestLookingShipAward: int
- positionScores: int[]

positionScores: int[]
BestLookingShipAward: int

Screen

+ Screen():

+ notifyRequestRandComponent(ShipBoard, Component): void
+ notifyStashComponent(ShipBoard, Component): void
+ notifyCurrentPlayerUpdate(ShipBoard): void
+ notifyRemoveLobby(UUID): void
+ notifyPlaceComponent(ShipBoard, Point, Direction): void
+ notifyRemoveGoods(ShipBoard, Point, GoodsType): void
+ notifyPlayerJoin(Player): void
+ notifyReleaseForecast(ShipBoard, int): void
+ notifyChooseShipPiece(ShipBoard, int, List<Point>): void
+ notifyPlaceGoods(ShipBoard, Point, GoodsType): void
+ notifyGrabStashedComponent(ShipBoard, int, Component): void
+ notifyFlipHourglass(ShipBoard): void
+ notifyRequestComponent(ShipBoard, Component): void
+ notifyDrawCard(AdventureCard): void
+ notifyUseBattery(ShipBoard, Point): void
+ notifyPeekForecast(ShipBoard, int): void
+ notifyShipValidated(ShipBoard): void
+ notifyChoosePlanet(ShipBoard, int, ShipBoard): void
+ notifyComponentChange(ShipBoard, Point): void
+ notifyRejectComponent(ShipBoard, Component): void
+ notifyStashComponent(ShipBoard, Component, Point): void
+ notifyPlaceComponent(ShipBoard, Point, Direction, Point): void
+ notifyNewLobby(Lobby): void
+ notifyShipNotConnected(ShipBoard, List<Set<Point>>): void
+ notifyGiveUp(Player): void
+ notifyRemoveComponent(ShipBoard, Point): void
+ notifyFlightBoardPosition(ShipBoard, int): void
+ notifyActivateComponent(ShipBoard, Point): void
+ notifyHourglassEnd(): void
+ notifyLoseCrew(ShipBoard, Point): void
+ notifyGrabReward(ShipBoard, boolean): void
+ notifyInitializeCabin(ShipBoard, Point, CrewType, int): void
+ notifyGrabPlacedComponent(ShipBoard, Point): void
+ notifyRejectComponent(ShipBoard, Component, Point): void

forecastDeck: List<AdventureCard>
finalScores: Map<Player, Integer>

<<interface>>
ScreenFactory

+ createScreen(MetaState, ClientModel, ClientControllerInterface): S
+ createScreen(GameState, ClientModel, ClientControllerInterface): S

SecondDeck

+ SecondDeck(Game):

+ getForecastDeck(int): List<AdventureCard>
+ initMasterDeck(): void

SecondFactory

+ SecondFactory():

+ createScoresRegistry(): ScoresRegistry
+ createDeck(Game): Deck
+ createShipBoard(GameColor, GameEventListener): ShipBoard
+ createShipBuildingState(): ShipBuildingState
+ createShipCorrectionState(): ShipCorrectionState
+ createFlightBoard(int, GameEventListener): FlightBoard
+ createSurrenderPolicy(GameEventListener): SurrenderPolicy

SecondFactory

+ SecondFactory():

+ createShipBuildingState(): ShipBuildingState
+ createFlightBoard(): FlightBoard
+ createShipBoard(GameColor): ShipBoard

SecondFlightBoard

+ SecondFlightBoard(int, GameEventListener):

# loopLength: int

+ placeShipOnFlightBoard(ShipBoard, int): void
+ removeShips(Set<ShipBoard>): void

lappedShips: Set<ShipBoard>
loopLength: int

SecondShipBoard

+ SecondShipBoard(GameColor):

- stashedComponents: List<Component>
- shipArea: Set<Point>

+ initializeCabin(Point, CrewType): int
+ finishBuilding(): void
+ grabStashedComponent(int): void
+ loseCrew(Point): void
- addAliens(CrewType): void
# add(Point, Cabin): void
+ stashComponent(): void
# remove(Point, Cabin): void
- removeAliens(CrewType): void

enginePower: int
stashedComponents: List<Component>
firePower: int
shipArea: Set<Point>

SecondShipBoard

+ SecondShipBoard(GameColor, GameEventListener):

- stashedComponents: List<Component>
- lifeSupports: Map<Point, LifeSupport>

+ rejectComponent(): Component
# containsPoint(Point): boolean
+ initializeCabin(Point, CrewType): void
# add(LifeSupport, Point): void
+ getCrewTypeOptions(Point): Set<CrewType>
+ stashComponent(): void
+ remove(LifeSupport, Point): void
+ offerComponent(Component): void
# remove(Cabin, Point): void
+ grabStashedComponent(int): void
+ finishBuilding(): void
+ loseCrew(Point): void

enginePower: int
stashedComponents: List<Component>
firePower: int
lifeSupports: Map<Point, LifeSupport>

<<record>>
SecondShipBuildingDTO

+ SecondShipBuildingDTO(BuildingDataDTO, HourglassDTO, Map<Strin

+ blockedForecasts(): Map<String, Integer>
+ baseData(): BuildingDataDTO
+ h l () H l DTO

SecondShipBuildingState

+ SecondShipBuildingState():

- shipToForecasts: Map<ShipBoard, Integer>
- blockedForecasts: Set<Integer>
- hourglass: Hourglass

+ grabStashedComponent(ShipBoard, int): void
+ releaseForecast(ShipBoard): void
+ stashComponent(ShipBoard): void
# endBuilding(): void
+ acquireForecast(ShipBoard, int): void
+ placeShipOnFlightBoard(ShipBoard, int): void
+ flipHourglass(ShipBoard): void
# notifyHourglassEnd(): void

shipToForecasts: Map<ShipBoard, Integer>
game: Game
blockedForecasts: Set<Integer>
hourglass: Hourglass

SecondShipBuildingState

+ SecondShipBuildingState():

- hourglass: Hourglass
- blockedForecasts: ShipBoard[]

+ notifyStashComponent(ShipBoard): void
+ notifyPeekForecast(ShipBoard, int): void
+ notifyGrabStashedComponent(ShipBoard, int): void
+ notifyHourglassEnd(): void
+ notifyFlipHourglass(ShipBoard): void
+ leave(): void
+ notifyReleaseForecast(ShipBoard, boolean): void

availableActions: List<StateActions>
hasForecastDeck: boolean
blockedForecasts: ShipBoard[]
forecastDeck: List<AdventureCard>
hourglass: Hourglass

Server

+ Server(String, int, int, boolean, boolean):

+ main(String[]): void
+ start(): void

ServerController

+ ServerController(GameModelInterface):
+ ServerController(GameModelInterface, EventQueue<ControllerEvent>
+ ServerController(GameModelInterface, boolean, boolean):

- idToLobby: Map<UUID, Lobby>

- requestActiveLobbies(Player, boolean): void
- removeLobby(Lobby): void
+ handlePlayerDisconnection(Player): void
+ joinLobby(Player, UUID): LobbyInterface
+ leaveLobby(Player): void
+ newGame(Player, Level, int): LobbyInterface
+ registerNickname(String, ClientHandler): void

idToLobby: Map<UUID, Lobby>

<<interface>>
ServerControllerInterface

+ newGame(Player, Level, int): LobbyInterface
+ registerNickname(String, ClientHandler): void
+ joinLobby(Player, UUID): LobbyInterface
+ leaveLobby(Player): void
+ handlePlayerDisconnection(Player): void

<<interface>>
ServerHandler

+ dropConnection(): void
+ reconnect(): boolean

<<interface>>
ServerToClientInterface

+ signalDisconnection(): void
+ notifyEvent(Event): void

Session

+ Session(ClientHandler, Instant):

- lastPing: Instant
- clientHandler: ClientHandler

+ isExpired(Instant): boolean

clientHandler: ClientHandler
lastPing: Instant

SessionManager

- SessionManager():

- serverController: ServerControllerInterface
- instance: SessionManager

+ clear(): void
+ isPlayerActive(Player): boolean
- cleanup(): void
+ getClient(Player): ClientHandler
+ shutDown(): void
+ ping(Player): void
+ registerClient(Player, ClientHandler): void
+ unregisterClient(Player): void

serverController: ServerControllerInterface
instance: SessionManager

<<record>>
SetActiveLobbiesEvent

+ SetActiveLobbiesEvent(String, List<ActiveLobbyDTO>, boolean):

+ activeLobbies(): List<ActiveLobbyDTO>
+ reconnect(): boolean
+ playerName(): String
+ shouldResume(): boolean

SetupUtils

+ SetupUtils():

+ setupComponent(ComponentDTO): Component
+ setupShipBoard(ShipBoard, ShipBoardDTO): void

Shield

+ Shield(Map<Direction, Connector>, int):

- active: boolean

active: boolean

Shield

+ Shield(Map<Direction, Connector>):
+ Shield(Map<Direction, Connector>, int):

- active: boolean

+ activate(): void
+ deactivate(): void

active: boolean
defensibleDirections: Set<Direction>

ShipBoard

+ ShipBoard(GameColor, GameEventListener):

# firePower: int
# losses: int
# numBatteries: int
# credits: int
# componentMap: Map<Point, Component>
# shieldDirections: Set<Direction>
# cannons: Map<Point, Cannon>
# goods: Map<GoodsType, Integer>
# lastComponent: Component
# batteries: Map<Point, Battery>
# engines: Map<Point, Engine>
# crewSize: int
# activatables: Map<Point, Activatable>
# lastPosition: Point
# cargoHolds: Map<Point, CargoHold>
# enginePower: int
# cabins: Map<Point, Cabin>
# center: Point
# shields: Map<Point, Shield>

+ weldLastComponent(): void
+ offerComponent(Component): void
+ loseCrew(Point): void
+ discardComponent(Point): void
# containsPoint(Point): boolean
- add(Cannon, Point): void
- remove(CargoHold, Point): void
+ removeAll(boolean): void
- add(Component): void
- add(Cabin, Point): void
# remove(Cabin, Point): void
- add(Shield, Point): void
- deactivate(Shield): void
- removeComponent(Point, boolean): void
+ deactivateComponent(Point): void
+ removeGoods(Point, GoodsType): void
- activate(Shield): void
+ gainCredits(int): void
- remove(Engine, Point): void
+ placeGoods(Point, GoodsType, int): void
- deactivate(DoubleEngine): void
- add(DoubleEngine, Point): void
- activate(DoubleEngine): void
- add(Battery, Point): void
+ deactivateAll(): void
- remove(DoubleEngine, Point): void
- activate(DoubleCannon): void
+ stashComponent(): void
- remove(DoubleCannon, Point): void
- remove(Battery, Point): void
# getNeighbours(Point): Map<Direction, Point>
- add(Engine, Point): void
- add(DoubleCannon, Point): void
+ rejectComponent(): Component
+ activateComponent(Point): boolean
- add(CargoHold, Point): void
+ grabPlacedComponent(): void
+ finishBuilding(): void
- remove(Shield, Point): void
+ getCrewTypeOptions(Point): Set<CrewType>
# remove(LifeSupport, Point): void
+ placeComponent(Point, Direction): void
+ keepShipPiece(List<Set<Point>>, int, boolean): void
+ initializeCabin(Point, CrewType): void
+ checkValidity(): boolean
+ removeComponent(Point): void
+ useBatteries(Point): void
- deactivate(DoubleCannon): void
+ addWeldedComponent(Component, Point, Direction): void
+ grabStashedComponent(int): void
# add(LifeSupport, Point): void
+ setup(Map<Point, Component>, int, int): void
- remove(Cannon, Point): void

enginePower: int
exposedConnectorsNumber: int
connectedSets: List<Set<Point>>
stashedComponents: List<Component>
crewSize: int
losses: int
numBatteries: int
shieldDirections: Set<Direction>
engines: Map<Point, Engine>
batteries: Map<Point, Battery>
shields: Map<Point, Shield>
firePower: int
componentMap: Map<Point, Component>
cannons: Map<Point, Cannon>
goodsValue: int
lastPosition: Optional<Point>
goods: Map<GoodsType, Integer>
activatables: Map<Point, Activatable>
cabins: Map<Point, Cabin>
lastComponent: Optional<Component>
center: Point
credits: int
cargoHolds: Map<Point, CargoHold>
lifeSupports: Map<Point, LifeSupport>

ShipBoard

+ ShipBoard(GameColor):

# enginePower: int
# cabins: Map<Point, Cabin>
# crewSize: int
# losses: int
# credits: int
# cannons: Map<Point, Cannon>
# color: GameColor
# shields: Map<Point, Shield>
# lastComponent: Component
# firePower: int
# activatables: Map<Point, Activatable>
# batteries: Map<Point, Battery>
# cargoHolds: Map<Point, CargoHold>
# lastPosition: Point
# engines: Map<Point, Engine>
# cargoOnShip: int
# componentMap: Map<Point, Component>
# numBatteries: int

- updateStats(Point, Component): void
+ weldLastComponent(): void
+ finishBuilding(): void
+ activateComponent(Point): void
+ deactivateAll(): void
# resetLastComponent(): void
+ deactivateComponent(Point): void
+ removeComponent(Point): void
+ placeComponent(Point, Direction): void
+ removeShipPiece(List<Set<Point>>, int): List<Point>
+ grabStashedComponent(int): void
+ incrementLosses(int): void
+ setHand(Component, Point): void
+ stashComponent(): void
+ removeGoods(Point, GoodsType): void
+ loseCrew(Point): void
+ offerComponent(Component): void
+ initializeCabin(Point, CrewType): int
+ grabPlacedComponent(): void
+ rejectComponent(): Component
+ useBattery(Point): void
+ placeGoods(Point, GoodsType): void
# add(Point, Cabin): void
# remove(Point, Cabin): void

enginePower: int
stashedComponents: List<Component>
crewSize: int
losses: int
numBatteries: int
engines: Map<Point, Engine>
batteries: Map<Point, Battery>
shields: Map<Point, Shield>
firePower: int
componentMap: Map<Point, Component>
cannons: Map<Point, Cannon>
activatables: Map<Point, Activatable>
cabins: Map<Point, Cabin>
shipArea: Set<Point>
lastPosition: Point
credits: int
cargoOnShip: int
lastComponent: Component
cargoHolds: Map<Point, CargoHold>
lifeSupports: Map<Point, LifeSupport>
color: GameColor

<<record>>
ShipBoardDTO

+ ShipBoardDTO(Map<Point, ComponentDTO>, int, Point, List<Integ

+ losses(): int
+ lastComponent(): int
+ credits(): int
+ stashedComponents(): List<Integer>
+ lastPosition(): Point
+ componentMap(): Map<Point ComponentDTO>

<<record>>
ShipBuildingDTO

+ ShipBuildingDTO():

ShipBuildingState

+ ShipBuildingState():

# completedShipBoards: Set<ShipBoard>
- componentBank: ComponentBank

+ notifyRequestRandComponent(ShipBoard, Component): void
+ notifyPlaceComponent(ShipBoard, Point, Direction): void
+ notifyFlightBoardPosition(ShipBoard, int, boolean): void
+ notifyRequestComponent(ShipBoard, Component): void
+ notifyRejectComponent(ShipBoard): void
+ notifyGrabPlacedComponent(ShipBoard): void
+ componentInHand(): boolean
+ leave(): void

availableActions: List<StateActions>
componentBank: ComponentBank
completedShipBoards: Set<ShipBoard>

ShipBuildingState

+ ShipBuildingState():
# ShipBuildingState(ComponentBank):

# pendingShipBoards: Set<ShipBoard>
- componentBank: ComponentBank
# completedShipBoards: Set<ShipBoard>

- tryToEnd(): void
+ requestRandComponent(ShipBoard): void
# completeShipBoard(ShipBoard): void
+ rejectComponent(ShipBoard): void
+ cancelSkip(ShipBoard): void
+ placeComponent(ShipBoard, Point, Direction): void
+ flipHourglass(ShipBoard): void
+ grabPlacedComponent(ShipBoard): void
+ releaseForecast(ShipBoard): void
+ stashComponent(ShipBoard): void
+ placeShipOnFlightBoard(ShipBoard): void
+ skip(ShipBoard): void
# endBuilding(): void
+ grabStashedComponent(ShipBoard, int): void
- removePending(ShipBoard): void
+ requestComponent(ShipBoard, int): void
+ placeShipOnFlightBoard(ShipBoard, int): void
+ acquireForecast(ShipBoard, int): void

componentBank: ComponentBank
completedShipBoards: Set<ShipBoard>
pendingShipBoards: Set<ShipBoard>

<<record>>
ShipCorrectionDTO

+ ShipCorrectionDTO(Set<String>, Map<String, List<Set<Point>>>, boo

+ validShips(): Set<String>
+ shipPieces(): Map<String, List<Set<Point>>>
+ h ldDi d() b l

ShipCorrectionState

+ ShipCorrectionState(ShipBoard, Set<ShipBoard>, Map<ShipBoard, L

- shipPieces: Map<ShipBoard, List<Set<Point>>>
- validShipBoards: Set<ShipBoard>

+ notifyChooseShipPiece(ShipBoard, int): void
+ notifyInitializeCabin(ShipBoard, Point, CrewType): void
+ notifyShipNotConnected(ShipBoard, List<Set<Point>>): void
+ notifyRemoveComponent(ShipBoard, Point): void
+ notifyShipValidated(ShipBoard): void

availableActions: List<StateActions>
shipPieces: Map<ShipBoard, List<Set<Point>>>
validShipBoards: Set<ShipBoard>

ShipCorrectionState

+ ShipCorrectionState(boolean):

- pendingShipBoards: Set<ShipBoard>
- shouldDiscard: boolean
- validShipBoards: Set<ShipBoard>
- shipPiecesMap: Map<ShipBoard, List<Set<Point>>>

- checkShipConnection(ShipBoard): boolean
- tryStateTransition(): void
+ cancelSkip(ShipBoard): void
+ getShipPieces(ShipBoard): List<Set<Point>>
+ chooseShipPiece(ShipBoard, int): void
+ removeComponent(ShipBoard, Point): void
+ skip(ShipBoard): void
- removeAt(ShipBoard, Point): void
- defaultAction(ShipBoard): void
- checkShipValidity(ShipBoard): boolean

shouldDiscard: boolean
game: Game
validShipBoards: Set<ShipBoard>
shipPiecesMap: Map<ShipBoard, List<Set<Point>>>
pendingShipBoards: Set<ShipBoard>

<<record>>
ShipInitializationDTO

+ ShipInitializationDTO(Map<String, Map<CrewType, Set<Point>>>):

+ crewTypeToCabins(): Map<String, Map<CrewType, Set<Point>>>

ShipInitializationState

+ ShipInitializationState(ShipBoard, Map<ShipBoard, Map<CrewType, Set<

- crewtypeToPoints: Map<ShipBoard, Map<CrewType, Set<Point>>>

+ notifyInitializeCabin(ShipBoard, Point, CrewType): void

availableActions: List<StateActions>
crewtypeToPoints: Map<ShipBoard Map<CrewType Set<Point>>>

ShipInitializationState

+ ShipInitializationState():

- pendingShipBoards: Set<ShipBoard>
- shipRelevantCabins: Map<ShipBoard, Map<CrewType, Set<Point>>>

+ initializeCabin(ShipBoard, Point, CrewType): void
+ getCrewTypeMap(ShipBoard): Map<CrewType, Set<Point>>
- tryStateTransition(): void
+ cancelSkip(ShipBoard): void
+ goNext(ShipBoard): void
- initHumans(ShipBoard, boolean): void
+ skip(ShipBoard): void

shipRelevantCabins: Map<ShipBoard, Map<CrewType, Set<Point>>>
game: Game
pendingShipBoards: Set<ShipBoard>

<<record>>
ShipNotConnectedEvent

+ ShipNotConnectedEvent(String, List<Set<Point>>):

+ playerName(): String
+ shipPieces(): List<Set<Point>>

<<record>>
ShipPieceRemoveEvent

+ ShipPieceRemoveEvent(String, int):

+ playerName(): String
+ index(): int

<<record>>
SimpleStateDTO

+ SimpleStateDTO(String, StateDTOType):

+ type(): StateDTOType
+ playerName(): String

SlaversCard

+ SlaversCard(Game, Level, int, int, int, int, int):

+ getReward(): void
+ initialize(): void

nextState: AdventureState

SlaversCard

+ SlaversCard(Level, int, int, int, int, int):

- creditPrize: int
- flightDaysLoss: int
- firePowerThreshold: int
- crewLoss: int

creditPrize: int
firePowerThreshold: int
flightDaysLoss: int
crewLoss: int

SmallFire

+ SmallFire(Direction):
+ SmallFire(IntSupplier, Direction):

+ getActivatablePoints(ShipBoard): Set<Point>
# getComponentPositionToRemove(ShipBoard): Optional<Point>

projectileType: ProjectileType

SmallMeteor

+ SmallMeteor(IntSupplier, Direction):
+ SmallMeteor(Direction):

+ getActivatablePoints(ShipBoard): Set<Point>
# getComponentPositionToRemove(ShipBoard): Optional<Point>

projectileType: ProjectileType

SmugglersCard

+ SmugglersCard(Game, Level, int, int, Map<GoodsType, Integer>,

+ getReward(): void
+ initialize(): void

nextState: AdventureState

SmugglersCard

+ SmugglersCard(Level, int, int, Map<GoodsType, Integer>, int, int)

- goodsPrize: Map<GoodsType, Integer>
- goodsLoss: int
- firePowerThreshold: int
- flightDaysLoss: int

firePowerThreshold: int
flightDaysLoss: int
goodsLoss: int
goodsPrize: Map<GoodsType, Integer>

SocketClient

+ SocketClient():

- controller: ServerToClientInterface

+ requestComponent(int): void
+ releaseForecast(): void
+ leaveLobby(): void
+ drawCard(): void
+ goNext(): void
+ start(String, int): void
+ joinLobby(UUID): void
+ placeComponent(Point, Direction): void
+ chooseShipPiece(int): void
+ placeGoods(Point, GoodsType): void
+ rejectComponent(): void
+ loseGoods(Point): void
- inputThreadTask(): void
+ reconnect(): boolean
+ grabReward(): void
+ grabPlacedComponent(): void
+ removeGoods(Point, GoodsType): void
+ choosePlanet(int): void
+ grabStashedComponent(int): void
+ placeShipOnFlightBoard(): void
+ useBattery(Point): void
+ stashComponent(): void
+ requestRandComponent(): void
+ giveUp(): void
+ stop(): void
+ dropConnection(): void
- handleIOException(IOException): void
- ping(): void
+ placeShipOnFlightBoard(int): void
+ removeComponent(Point): void
+ notifyEvent(Event): void
- sendRequest(Request): void
- connect(): void
+ registerNickname(String): void
+ acquireForecast(int): void
+ requestNewGame(Level, int): void
+ initializeCabin(Point, CrewType): void
+ activateComponent(Point): void
+ flipHourglass(): void
+ loseCrew(Point): void

controller: ServerToClientInterface

SocketClientHandler

+ SocketClientHandler(SafeSocket, ServerControllerInterface):

- player: Player

+ acquireForecast(int): void
+ requestNewGame(Level, int): void
- runRequest(Request): Response
+ placeShipOnFlightBoard(): void
+ useBattery(Point): void
+ placeShipOnFlightBoard(int): void
+ releaseForecast(): void
+ goNext(): void
+ giveUp(): void
+ pauseEvents(): void
+ grabStashedComponent(int): void
+ initializeCabin(Point, CrewType): void
+ chooseShipPiece(int): void
+ drawCard(): void
+ requestComponent(int): void
+ placeComponent(Point, Direction): void
+ joinLobby(UUID): void
+ leaveLobby(): void
+ grabReward(): void
+ registerNickname(String): void
+ flipHourglass(): void
+ stashComponent(): void
+ removeComponent(Point): void
+ grabPlacedComponent(): void
+ stop(): void
- updateTask(): void
- handleIOException(IOException): void
- requestTask(): void
+ requestRandComponent(): void
+ loseGoods(Point): void
+ start(): void
+ activateComponent(Point): void
+ loseCrew(Point): void
+ removeGoods(Point, GoodsType): void
+ choosePlanet(int): void
+ notifyEvent(Event): void
+ rejectComponent(): void
+ placeGoods(Point, GoodsType): void

lobby: LobbyInterface
player: Player

SocketServer

+ SocketServer(ServerControllerInterface):

- listen(): void
+ start(int): void
- handleIOException(IOException): void
+ stop(): void

StarDustCard

+ StarDustCard(Game, Level, int):

+ initialize(): void

nextState: AdventureState

StarDustCard

+ StarDustCard(Level, int):

StashComponent

+ StashComponent():

+ execute(VirtualServer): void

<<record>>
StashComponentEvent

+ StashComponentEvent(String):

+ playerName(): String

<<enumeration>>
StateActions

+ StateActions():

+ valueOf(String): StateActions
+ values(): StateActions[]

<<interface>>
StateDTO

<<enumeration>>
StateDTOType

+ StateDTOType():

+ values(): StateDTOType[]
+ valueOf(String): StateDTOType

<<enumeration>>
SurrenderCause

+ SurrenderCause():

+ valueOf(String): SurrenderCause
+ values(): SurrenderCause[]

<<record>>
SurrenderEvent

+ SurrenderEvent(List<String>):

+ playerNames(): List<String>

<<interface>>
SurrenderPolicy

+ requestSurrender(ShipBoard, SurrenderCause): boolean
+ confirmSurrender(FlightBoard): Set<ShipBoard>

surrenderedShips: Set<ShipBoard>
d E bl d b l

<<record>>
SurrenderRequestEvent

+ SurrenderRequestEvent(String, SurrenderCause):

+ cause(): SurrenderCause
+ playerName(): String

TestDeck

+ TestDeck(Game):

TestFactory

+ TestFactory():

+ createShipBuildingState(): ShipBuildingState
+ createFlightBoard(int, GameEventListener): FlightBoard
+ createShipBoard(GameColor, GameEventListener): ShipBoard
+ createShipCorrectionState(): ShipCorrectionState
+ createSurrenderPolicy(GameEventListener): SurrenderPolicy
+ createScoresRegistry(): ScoresRegistry
+ createDeck(Game): Deck

TestFactory

+ TestFactory():

+ createShipBoard(GameColor): ShipBoard
+ createFlightBoard(): FlightBoard
+ createShipBuildingState(): ShipBuildingState

TestFlightBoard

+ TestFlightBoard(int, GameEventListener):

# loopLength: int

loopLength: int

TestShipBoard

+ TestShipBoard(GameColor, GameEventListener):

# containsPoint(Point): boolean
+ discardComponent(Point): void

TestShipBoard

+ TestShipBoard(GameColor):

- shipArea: Set<Point>

shipArea: Set<Point>

<<record>>
TestShipBuildingDTO

+ TestShipBuildingDTO(BuildingDataDTO):

+ data(): BuildingDataDTO

TestShipBuildingState

+ TestShipBuildingState():

availableActions: List<StateActions>

TestShipBuildingState

+ TestShipBuildingState():

# endBuilding(): void

UseBattery

+ UseBattery(Point):

+ execute(VirtualServer): void

<<record>>
UseBatteryEvent

+ UseBatteryEvent(String, Point):

+ playerName(): String
+ point(): Point

<<record>>
ValidateShipEvent

+ ValidateShipEvent(String):

+ playerName(): String

View

# View(ClientModel, ClientController, ScreenFactory<S>):

<<interface>>
VirtualClient

+ notifyEvent(Event): void

<<interface>>
VirtualServer

+ grabPlacedComponent(): void
+ placeShipOnFlightBoard(int): void
+ leaveLobby(): void
+ stashComponent(): void
+ joinLobby(UUID): void
+ initializeCabin(Point, CrewType): void
+ useBattery(Point): void
+ flipHourglass(): void
+ registerNickname(String): void
+ grabReward(): void
+ acquireForecast(int): void
+ drawCard(): void
+ loseGoods(Point): void
+ loseCrew(Point): void
+ rejectComponent(): void
+ requestNewGame(Level, int): void
+ placeComponent(Point, Direction): void
+ requestRandComponent(): void
+ removeGoods(Point, GoodsType): void
+ giveUp(): void
+ removeComponent(Point): void
+ chooseShipPiece(int): void
+ requestComponent(int): void
+ placeGoods(Point, GoodsType): void
+ grabStashedComponent(int): void
+ choosePlanet(int): void
+ releaseForecast(): void
+ placeShipOnFlightBoard(): void
+ activateComponent(Point): void
+ goNext(): void